

Bonus Active and Inactive data

```
SELECT
  t.user_id,
  t.user_status,
  COUNT(*) AS scanned_boxes,
  ROUND(SUM(t.points) * 0.5) AS bonus_points
FROM
  (
    SELECT
      u.id AS user_id,
      mp.points,
      CASE
        WHEN EXISTS (
          SELECT 1
          FROM sku_inventories si2
          WHERE si2.status_retailer_id = u.id
          AND si2.retailer_scanned_at BETWEEN
            '2025-11-01 00:00:00' AND '2026-01-31 23:59:59'
        )
        THEN 'Active'
        ELSE 'Inactive'
      END AS user_status,
      ROW_NUMBER() OVER (
        PARTITION BY u.id
        ORDER BY si.retailer_scanned_at ASC
      ) AS rn
    FROM users u
    JOIN sku_inventories si
      ON si.status_retailer_id = u.id
      AND si.retailer_scanned_at BETWEEN
        '2026-02-01 00:00:00' AND '2026-02-28 23:59:59'
    JOIN mypoints mp
      ON mp.sku_inventory_id = si.id
      AND mp.users_id = si.status_retailer_id
    WHERE u.user_role = 2
    AND u.created_at < '2025-11-01'
  ) t
WHERE t.rn <= 10
GROUP BY t.user_id, t.user_status
HAVING COUNT(*) >= 10;
```

#adepoli Earning

```
SELECT
    user_id,
    scheme_id,
    SUM(amount) AS balance,

    SUM(CASE
        WHEN reference_type = 'sku_scan'
        THEN amount
        ELSE 0
    END) AS earning,

    abs(SUM(CASE
        WHEN reference_type = 'scheme_gift_order'
        THEN amount
        ELSE 0
    END)) AS redemption
FROM scheme_wallet_transactions w
join schemes s on
w.scheme_id = s.id
WHERE w.scheme_id = 1
GROUP BY user_id, scheme_id
order by balance;
```

#winter Earning

```
SELECT
    DISTINCT user_id,
    SUM(amount) AS balance,
    SUM(CASE
        WHEN reference_type IN ('sku_scan','topup')
        THEN amount
        ELSE 0
    END) AS Earning,
    abs(SUM(CASE
        WHEN reference_type IN ('gift_order','cash_withdrawal','credit_note')
        THEN amount
        ELSE 0
    END)) AS Redemption
```

```

FROM winter_scan_wallet_transactions
GROUP BY
    user_id
ORDER BY
    balance;

```

#namma & majja Earning

```

SELECT
    user_id,
    SUM(amount) AS balance,
    SUM(CASE
        WHEN reference_type IN ('sku_scan','topup')
        THEN amount
        ELSE 0
    END) AS Earning,
    Abs(SUM(CASE
        WHEN reference_type IN ('gift_order','credit_note')
        THEN amount
        ELSE 0
    END)) AS Redemption
FROM gift_scan_wallet_transactions
GROUP BY
    user_id
ORDER BY
    balance;

```

#freedom Earning

```

SELECT
    u.id,
    COALESCE(pb.prev_balance, 0) AS prev_balance,
    COALESCE(w.earn_freedom_balance, 0) AS earning,
    COALESCE(r.redeemed, 0) AS redemption,
    sum(wt.amount) as current_amount,
    ABS(
        CASE
            WHEN COALESCE(pb.prev_balance, 0) - COALESCE(wd.withdrawn, 0) >= 0
            THEN 0
            ELSE COALESCE(pb.prev_balance, 0) - COALESCE(wd.withdrawn, 0)
        END
    ) AS freedom_cash_withdrawal,
    (
        COALESCE(w.earn_freedom_balance, 0)

```

```

- COALESCE(r.redeemed, 0)
+
CASE
  WHEN COALESCE(pb.prev_balance, 0)
    - COALESCE(wd.withdrawn, 0) >= 0
  THEN 0
  ELSE COALESCE(pb.prev_balance, 0)
    - COALESCE(wd.withdrawn, 0)
  END
) AS freedom_balance
FROM users u
LEFT JOIN
(
  SELECT user_id,
    SUM(amount) AS earn_freedom_balance
  FROM wallet_transaction
  WHERE reference_type IN ('cash_point', 'topup')
    AND created_at BETWEEN '2025-09-01 00:00:00'
      AND '2025-11-20 23:59:59'
  GROUP BY user_id
) w ON w.user_id = u.id
left join

wallet_transaction wt on
wt.user_id = u.id

LEFT JOIN
(
  SELECT go.user_id,
    SUM(go.used_points) AS redeemed
  FROM gift_orders go
  INNER JOIN gift_cards gc3
    ON gc3.id = go.gift_card_id
  WHERE go.order_date BETWEEN '2025-09-01 00:00:00'
    AND '2025-11-20 23:59:59'
    AND gc3.points_applicability = 'date_range'
    AND go.status != 'Cancelled'
  GROUP BY go.user_id
) r ON r.user_id = u.id
LEFT JOIN
(
  SELECT user_id,
    SUM(amount) AS prev_balance
  FROM wallet_transaction
  WHERE created_at < '2025-09-01 00:00:00'
  GROUP BY user_id
) pb ON pb.user_id = u.id
LEFT JOIN
(

```

```

SELECT user_id,
       SUM(amount) AS withdrawn
FROM withdrawal_request
WHERE created_at BETWEEN '2025-09-01 00:00:00'
      AND '2025-11-20 23:59:59'
      AND withdraw_request_type IS NULL
      AND status IN (0,1)
GROUP BY user_id
) wd ON wd.user_id = u.id
WHERE u.user_role = 2
      AND u.state_id IN (8, 9, 20, 30, 33, 34, 40)
GROUP BY u.id;

```

#Distributor_wallet_data

```

SELECT
  c.id AS distributor_id,
  c.sap_code,
  COALESCE(w.wallet_amount,0) AS wallet_amount,
  COALESCE(d.deactivated_wallet_amount,0) AS deactivated_wallet_amount
FROM users u
JOIN companies c
  ON u.distributer_id = c.id
LEFT JOIN (
  SELECT user_id, SUM(amount) AS wallet_amount
  FROM wallet_transaction
  GROUP BY user_id
) w
  ON w.user_id = u.id
LEFT JOIN (
  SELECT user_id, SUM(amount) AS deactivated_wallet_amount
  FROM deactivated_qr_wallet_transactions
  GROUP BY user_id
) d
  ON d.user_id = u.id
WHERE u.user_role = 4;

```

#check the QR scan

```
SELECT *
FROM sku_qr_points_maps
WHERE sku_code IN
('ASI025BRN#100','ASI025BRN#85','ASI026BRN#95','ASI025BRN#95','ASI026BRN#95');
```

#Check the bank Transactions data for users Transactions

```
SELECT *
FROM withdrawal_request wt
JOIN automatic_transactions ats
    ON wt.user_id = ats.user_id
JOIN bank_transactions bt
    ON bt.bankReferenceNumber = ats.bank_reference_number
WHERE wt.user_id = 73502;
```

```
SELECT *
FROM withdrawal_request
where user_id = 73502;
```

```
select * from automatic_transactions where transaction_id = '9FBA22CC952B4E2FBC92C23EB9F2D05B';
```

```
select * from bank_transactions where recieverAccountNumber = 03481000103345;
```

```
select * from bank_accounts where user_id = 73502;
```

#Box scan count of specific retailer

```
select count(uom) from sku_inventories where uom in ('B10','B5')
and status_retailer_id = 48683 and Month(retailer_scanned_at) = 2;
```

Check the wholesaler and distributor report winter and core till march 2026

winter - wholesaler

```
SELECT
    si.status_wholeseller_id,
    SUM(wswt.amount) AS total_amount
FROM sku_inventories si
JOIN sku_products sp
    ON si.product_id = sp.id
JOIN winter_scan_wallet_transactions wswt
    ON si.status_wholeseller_id = wswt.id
WHERE sp.season IN ('winter')
and Date(wholesaler_scanned_at) < '2026-04-01'
GROUP BY si.status_wholeseller_id;
```

#core - wholesaler

```
SELECT
    si.status_wholeseller_id,
    SUM(wswt.amount) AS total_amount
FROM sku_inventories si
JOIN sku_products sp
    ON si.product_id = sp.id
JOIN winter_scan_wallet_transactions wswt
    ON si.status_wholeseller_id = wswt.id
WHERE sp.season IN ('core')
and Date(wholesaler_scanned_at) < '2026-04-01'
GROUP BY si.status_wholeseller_id;
```

#coreDistributor__dwt

#coreDistributor__dwt

```
select u.distributor_id,  
sum(wt.amount) as Total_amount  
from users u  
join wallet_transaction wt on  
wt.user_id = u.id  
where u.user_role = 4  
and Date(wt.created_at) < '2026-04-01'  
group by u.distributor_id;
```

Check bank transaction for retailers

```
select sum(amount) from wallet_transaction where user_id = 74442 and reference_type IN(  
'topup','cash_point');
```

```
select sum(amount) from wallet_transaction where user_id = 74442;  
select * from wallet_transaction where user_id = 74442 and reference_type IN( 'topup');
```

```
select * from withdrawal_request where user_id = 74442;
```

```
select * from automatic_transactions where transaction_id = 'BBDEC3C0B1AB4C408B7359A958A82935';
```

```
select * from bank_transactions where decentroUrn = 'BBDEC3C0B1AB4C408B7359A958A82935';
```

Bonus_transactions_data - apr_nov and dec_mar

#apr_nov

```
SELECT  
u.id AS customer_id,  
md.shop_name,  
u.district,
```

u.pincode,

SUM(CASE WHEN MONTH(bt.created_at)=4 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS APR,
SUM(CASE WHEN MONTH(bt.created_at)=5 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS MAY,
SUM(CASE WHEN MONTH(bt.created_at)=6 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS JUN,
SUM(CASE WHEN MONTH(bt.created_at)=7 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS JUL,
SUM(CASE WHEN MONTH(bt.created_at)=8 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS AUG,
SUM(CASE WHEN MONTH(bt.created_at)=9 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS SEP,
SUM(CASE WHEN MONTH(bt.created_at)=10 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0
END) AS OCT,
SUM(CASE WHEN MONTH(bt.created_at)=11 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0
END) AS NOV

FROM users u

LEFT JOIN bonus_transactions bt
ON bt.user_id = u.id

LEFT JOIN bonus_events be
ON be.id = bt.bonus_event_id

LEFT JOIN mechanic_details md
ON u.id = md.mechanic_id

WHERE be.id IN (13,14,15,19,20,21,22) and u.user_role = 2

GROUP BY
u.id,
md.shop_name,
u.district,
u.pincode;

dec_mar

```

SELECT
u.id AS customer_id,
md.shop_name,
u.district,
u.pincode,

SUM(CASE WHEN MONTH(wt.created_at)=12 AND YEAR(wt.created_at)=2025 THEN wt.amount ELSE 0
END) AS `DEC`,
SUM(CASE WHEN MONTH(wt.created_at)=1 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0
END) AS JAN,
SUM(CASE WHEN MONTH(wt.created_at)=2 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0
END) AS FEB,
SUM(CASE WHEN MONTH(wt.created_at)=3 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0
END) AS MAR

FROM wallet_transaction wt

LEFT JOIN bonus_events be
    ON be.id = wt.reference_id

LEFT JOIN users u
    on u.id = wt.user_id

LEFT JOIN mechanic_details md
    ON wt.user_id = md.mechanic_id

WHERE be.id IN (13,14,15,19,20,21,22) and u.user_role = 2 and wt.reference_type = 'topup'

GROUP BY
u.id,
md.shop_name,
u.district,
u.pincode;

```

Checking bonus_transactions for specific date

```

select sum(points) from bonus_transactions
join bonus_events on

```

```
bonus_transactions.bonus_event_id = bonus_events.id
where bonus_events.id IN (13,14,15,19,20,21,22)
and MONTH(bonus_transactions.created_at) in (4,5,6,7,8,9,10,11)
and bonus_transactions.user_id =50565;
```

```
select sum(amount) from wallet_transaction
join bonus_events on
wallet_transaction.reference_id = bonus_events.id
where bonus_events.id IN (13,14,15,19,20,21,22)
and wallet_transaction.reference_type = 'topup'
and wallet_transaction.user_id = 50565;
```

Unscanned_data for Winter season

```
SELECT
    u.distributer_id,
    sk.lpn_number,
    sp.season,
    d.amount AS amount,
    sqpm.sku_code,
    sk.sku_description,
    sqpm.uom,
    sp.product_name,
    DATE(d.created_at) as Transaction_date
FROM deactivated_qr_wallet_transactions d
JOIN users u
    ON d.user_id = u.id
JOIN sku_inventories sk
    ON d.reference_id = sk.id
JOIN sku_products sp
    ON sk.product_id = sp.id AND sp.season = 'winter'
Join sku_qr_points_maps sqpm
on sk.sku_code = sqpm.sku_code
```

```
WHERE d.reference_type = 'sku_scan';
```

new retailer_category for the period jan-mar

```
SELECT rcsfeb.* FROM (
SELECT
    skui.status_retailer_id AS retailer_id,
    sp.categories as category_id,
    cat.name as category,

    SUM(
        CASE
            WHEN sqpm.box_calculation_uom IS NOT NULL
            AND sqpm.box_calculation_uom > 0
            THEN sqpm.box_calculation_uom

            WHEN sqpm.box_calculation_uom IS NULL
            OR sqpm.box_calculation_uom = 0
            THEN CASE
                WHEN skui.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) AS total_box_count

FROM sku_inventories skui
join sku_products sp on sp.id = skui.product_id
left join master_categories cat on cat.id = sp.categories
join users on users.id = skui.status_retailer_id
left JOIN sku_qr_points_maps sqpm
    ON sqpm.sku_code = skui.sku_code
```

AND sqpm.uom = skui.uom

where retailer_scanned_at between '2026-01-01 00:00:00' and '2026-03-12 23:59:59'
and users.created_at < '2026-01-01'

GROUP BY

skui.status_retailer_id,
sp.categories,
cat.name

) as rcsfeb

left join (

SELECT

skui.status_retailer_id AS retailer_id,
sp.categories as category_id,
cat.name as category,

SUM(
CASE
WHEN sqpm.box_calculation_uom IS NOT NULL
AND sqpm.box_calculation_uom > 0
THEN sqpm.box_calculation_uom

WHEN sqpm.box_calculation_uom IS NULL
OR sqpm.box_calculation_uom = 0
THEN CASE
WHEN skui.uom = 'B5' THEN 0.5
ELSE 1
END

END

) AS total_box_count

FROM sku_inventories skui

join sku_products sp on sp.id = skui.product_id

left join master_categories cat on cat.id = sp.categories

join users on users.id = skui.status_retailer_id

left JOIN sku_qr_points_maps sqpm

ON sqpm.sku_code = skui.sku_code

AND sqpm.uom = skui.uom

where retailer_scanned_at between '2025-04-01 00:00:00' and '2025-12-31 23:59:59'
and users.created_at < '2026-01-01'

GROUP BY

skui.status_retailer_id,
sp.categories,
cat.name

) as rjan on rjan.retailer_id = rcsfeb.retailer_id and
rjan.category_id = rcsfeb.category_id

where (rjan.total_box_count < 5 OR rjan.total_box_count IS NULL) and rc+sfeb.total_box_count >=5;

#Distributor_deactivated wallet amount data

SELECT

c.id AS distributor_id,
c.sap_code,
COALESCE(w.wallet_amount,0) AS wallet_amount,
COALESCE(d.deactivated_wallet_amount,0) AS deactivated_wallet_amount

FROM users u

JOIN companies c

ON u.distributer_id = c.id

LEFT JOIN (

SELECT user_id, SUM(amount) AS wallet_amount
FROM wallet_transaction
GROUP BY user_id

) w

ON w.user_id = u.id

LEFT JOIN (

```

SELECT user_id, SUM(amount) AS deactivated_wallet_amount
FROM deactivated_qr_wallet_transactions
GROUP BY user_id
) d
ON d.user_id = u.id

WHERE u.user_role = 4;

```

#bonus_Inactive_data for scanned and unscanned

```

SELECT
    t.user_id,
    t.user_status,
    COUNT(*) AS scanned_boxes,
    ROUND(SUM(t.points) * 0.5) AS bonus_points
FROM
(
    SELECT
        u.id AS user_id,
        mp.points,

        CASE
            WHEN EXISTS (
                SELECT 1
                FROM sku_inventories si2
                WHERE si2.status_retailer_id = u.id
                AND si2.retailer_scanned_at BETWEEN
                '2025-11-01 00:00:00' AND '2026-01-31 23:59:59'
            )
            THEN 'Active'
            ELSE 'Inactive'
        END AS user_status,

        ROW_NUMBER() OVER (
            PARTITION BY u.id
            ORDER BY si.retailer_scanned_at ASC
        ) AS rn

    FROM users u

    JOIN sku_inventories si
    ON si.status_retailer_id = u.id
    AND si.retailer_scanned_at BETWEEN
    '2026-02-01 00:00:00' AND '2026-02-28 23:59:59'

```

```

        JOIN mypoints mp
        ON mp.sku_inventory_id = si.id
        AND mp.users_id = si.status_retailer_id

        WHERE u.user_role = 2
        AND u.created_at < '2025-11-01'

    ) t

    WHERE t.rn <= 10
    GROUP BY t.user_id, t.user_status
    HAVING COUNT(*) >= 10;

```

#Welcome_bonus_data for Retailers

```

SELECT
    si.status_retailer_id,
    DATE(u.created_at) AS registered_at,
    COUNT(si.lpn_number) AS feb_box_count,
    MIN(DATE(si.retailer_scanned_at)) AS first_scan_date
FROM users u
JOIN sku_inventories si
    ON u.id = si.status_retailer_id

WHERE u.created_at <= '2025-10-31'

AND si.retailer_scanned_at >= '2026-02-01'
AND si.retailer_scanned_at < '2026-02-01'

AND u.id NOT IN (
    SELECT status_retailer_id
    FROM sku_inventories
    WHERE retailer_scanned_at >= '2025-11-01'
    AND retailer_scanned_at < '2026-02-01'
)

GROUP BY u.id, DATE(u.created_at)
HAVING COUNT(si.lpn_number) > 10;

```

#monthly_points_data and Monthly Avg Points

```
SELECT
    ma.user_id,
    SUM(sq.box_calculation_uom) AS total_boxes,
    ma.monthly_average
FROM sku_qr_points_maps sq
JOIN sku_inventories si
    ON sq.sku_code = si.sku_code
JOIN mypoints mp
    ON si.id = mp.sku_inventory_id
JOIN mapping_incentive_users mpu
    ON mp.users_id = mpu.user_id
JOIN monthly_averages ma
    ON ma.user_id = mpu.user_id
JOIN incentive_schemes ins
    ON ma.monthly_average BETWEEN ins.min_val AND ins.max_val
WHERE ins.status = 1
AND ins.start_date <= '2026-02-28'
AND ins.end_date >= '2026-02-01'
GROUP BY
    ma.user_id,
    ma.monthly_average
ORDER BY mpu.user_id;
```

-- winter wallet scan

```
SELECT
```

```

w.user_id,
sp.product_name,
si.sku_code,
si.sku_description as material_description,
si.uom,
si.distributer_id,
sp.product_code,
COUNT(si.lpn_number) AS total_lpn,
SUM(w.amount) AS total_amount,
DATE(w.created_at) AS scanned_date
FROM sku_inventories si
JOIN winter_scan_wallet_transactions w
  ON si.id = w.reference_id
  AND w.reference_type = 'sku_scan'
JOIN sku_products sp
  ON si.product_id = sp.id
WHERE sp.season = 'winter'
GROUP BY
  w.user_id,
  sp.product_name,
  si.sku_code,
  si.sku_description,
  si.uom,
  si.distributer_id,
  sp.product_code,
  DATE(w.created_at);

```

-- chat support crosscheck category name & id for specific retailer with distributor

```

SELECT
  c.id AS category_id,
  c.name as category_name
FROM sku_inventories si
LEFT JOIN sku_products sp
  ON si.product_id = sp.id
LEFT JOIN master_categories c
  ON c.id = sp.categories
WHERE si.status_retailer_id = 67417
AND si.distributer_id = 6318
AND date(si.retailer_scanned_at) >= '2025-07-01'
AND date(si.retailer_scanned_at) <= NOW()
GROUP BY c.id, c.name;

```

-- bonus point report

```
SELECT b.user_id,b.points, b.created_at, u.user_role
from bonus_transactions b join users u on b.user_id = u.id
where bonus_event_id = 18;
```

-- milestone Report

```
SELECT user_id ,reference_id, reference_type, created_at
from wallet_transaction
where reference_type = 'topup' and
created_at >= '2026-03-01' and
reference_id is not null;
```

-- Available Transaction / Balance

```
SELECT
    user_id,
    SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END) -
    SUM(CASE WHEN type = 'debit' THEN ABS(points) ELSE 0 END) AS balance
FROM bonus_transactions
WHERE created_at < '2026-03-01'
GROUP BY user_id;
```

-- first Scan Date

```
select status_retailer_id, min(date(retailer_scanned_at)) from sku_inventories where status_retailer_id
is not null group by status_retailer_id;
SELECT status_retailer_id, MIN(DATE(retailer_scanned_at))
FROM sku_inventories
WHERE status_retailer_id = '46678'
GROUP BY status_retailer_id;
```

```
select id, date(created_at) from users WHERE user_role = 2;
```

-- all combined (Retailer Master DB)

```
SELECT
    u.id AS user_id,
    DATE(u.created_at) AS register_date,
    wt.balance AS wallet_balance,
    bt.balance AS bonus_balance,
    si.first_scan_date
```

FROM users u

```
LEFT JOIN (  
  SELECT  
    user_id,  
    SUM(amount) AS balance  
  FROM wallet_transaction  
  WHERE created_at < '2026-03-01'  
  GROUP BY user_id  
) wt ON wt.user_id = u.id
```

```
LEFT JOIN (  
  SELECT  
    user_id,  
    SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END) -  
    SUM(CASE WHEN type = 'debit' THEN ABS(points) ELSE 0 END) AS balance  
  FROM bonus_transactions  
  WHERE created_at < '2026-03-01'  
  GROUP BY user_id  
) bt ON bt.user_id = u.id
```

```
LEFT JOIN (  
  SELECT  
    status_retailer_id,  
    MIN(DATE(retailer_scanned_at)) AS first_scan_date  
  FROM sku_inventories  
  WHERE status_retailer_id IS NOT NULL  
  GROUP BY status_retailer_id  
) si ON si.status_retailer_id = u.id
```

WHERE u.user_role = 2
and u.isBlacklisted = 0;

select sum(amount) from wallet_transaction where user_id = 46599
and created_at < '2026-03-01';

-- Reatailer basic list + state name from state table

select u.id, u.name, u.mobile_number, st.sname from users u left join state st on u.state_id = st.id
where u.user_role = 2;

-- Retailer Shop Name From MeCHANIC tABLE + uSERS

```
select u.id, m.shop_name from users u left join mechanic_details m on u.id = m.id
where u.user_role = 2;
```

-- Total Box Scanned for Distributer Report (Jan - Mar)

```
SELECT
    distributor_id,
    status_retailer_id,

    -- Monthly totals
    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 1 THEN
        CASE
            WHEN uom = 'B10' THEN 1
            WHEN uom = 'B5' THEN 0.5
            ELSE 0
        END
        ELSE 0
    END) AS jan_boxes,

    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 2 THEN
        CASE
            WHEN uom = 'B10' THEN 1
            WHEN uom = 'B5' THEN 0.5
            ELSE 0
        END
        ELSE 0
    END) AS feb_boxes,

    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 3 THEN
        CASE
            WHEN uom = 'B10' THEN 1
            WHEN uom = 'B5' THEN 0.5
            ELSE 0
        END
        ELSE 0
    END)
```

```

ELSE 0
END) AS mar_boxes,

-- Total for 3 months
SUM(
CASE
WHEN uom = 'B10' THEN 1
WHEN uom = 'B5' THEN 0.5
ELSE 0
END
) AS total_boxes_Jan_Mar

FROM sku_inventories

WHERE retailer_scanned_at >= '2026-01-01 00:00:00'
AND retailer_scanned_at < '2026-04-01 00:00:00'

GROUP BY distributer_id, status_retailer_id

ORDER BY distributer_id, status_retailer_id;

```

-- FINAL Winter distributor unscanned report

```

SELECT
    u.distributer_id,
    sk.lpn_number,
    sp.season,
    d.amount AS amount
FROM deactivated_qr_wallet_transactions d
JOIN users u
    ON d.user_id = u.id
JOIN sku_inventories sk
    ON d.reference_id = sk.id
JOIN sku_products sp
    ON sk.product_id = sp.id AND sp.season = 'winter'
WHERE d.reference_type = 'sku_scan'

```

-- to check cashpoint & bonus (top up)

```

select * from users where mobile_number = 9911570413;

SELECT *

```

```
FROM wallet_transaction
WHERE user_id = 53756
AND reference_type IN ('cash_point', 'topup')
AND created_at >= '2025-09-01'
AND created_at < '2026-04-15';
```

-- Total Earnings and Credit Note for Distributers

```
SELECT
    wt.user_id, sum(wt.amount) as Total_amount,
```

-- Total Earnings

```
    SUM(
    CASE
    WHEN wt.reference_type IN ('cash_point','topup','referral_earning','incentive')
    THEN wt.amount
    ELSE 0
    END
    ) AS Total_Earnings,
```

-- Credit Note

```
    SUM(
    CASE
    WHEN wt.reference_type = 'credit_note'
    THEN wt.amount
    ELSE 0
    END
    ) AS Credit_Note
```

```
FROM wallet_transaction wt
JOIN users u
    ON wt.user_id = u.id
```

```
WHERE
    wt.created_at < '2026-04-01' -- Till 31 st March
    AND u.user_role = 4
```

```
GROUP BY wt.user_id;
```

-- UOM BOX SCANNED

```
select COUNT(uom) from sku_inventories
where status_retailer_id = 75086
```

```
and uom = 'B5'  
and retailer_scanned_at >= '2026-04-01'  
and retailer_scanned_at < '2026-04-17';
```

```
select * from sku_inventories  
where status_retailer_id = 75086  
and uom in ('B5','B10');
```

-- Deactivated Unscanned Report for Distributer

```
SELECT  
    u.distributer_id,  
  
    SUM(  
        CASE  
            WHEN dqwt.reference_type IN ('sku_scan')  
            AND sp.season = 'winter'  
            THEN dqwt.amount  
            ELSE 0  
        END  
    ) AS Winter_Earning,  
    SUM(  
        CASE  
            WHEN dqwt.reference_type IN ('sku_scan')  
            AND ( sp.season = 'core' or sp.season is null)  
            THEN dqwt.amount  
            ELSE 0  
        END  
    ) AS core_Earning,  
  
    SUM(dqwt.amount) AS Balance,  
  
    SUM(  
        CASE  
            WHEN dqwt.reference_type IN ('topup')  
            THEN dqwt.amount  
            ELSE 0  
        END  
    ) AS Topup_Earning,  
    ABS (SUM(  
        CASE
```

```

        WHEN dqwt.reference_type IN ('credit_note') -- abs
        THEN dqwt.amount
        ELSE 0
        END
    ) )AS credit_note

FROM deactivated_qr_wallet_transactions dqwt

JOIN users u
    ON u.id = dqwt.user_id

LEFT JOIN sku_inventories si
    ON si.id = dqwt.reference_id

LEFT JOIN sku_products sp
    ON sp.id = si.product_id

WHERE dqwt.created_at < '2026-04-01'
AND u.user_role = 4
GROUP BY u.distributer_id;

```

-- Credit Note for retailer & wholesaler report

```

SELECT u.`name` as name,c.`name`as gift_name,
so.shop_name,so.order_date,so.used_points,so.status,so.order_id,so.user_id,so.cn_amount,so.cn_date
FROM scheme_gift_orders so
join gift_cards c on so. = c.id
join users u on so.user_id = u.id
WHERE so.status = 'CN adjusted'
and u.user_role in (2,5);

```

-- Distributer Unsacnned Wallet Point Yearly Report

```

SELECT
    u.distributer_id,
    DATE(dqwt.created_at) AS created_date,
    si.lpn_number,
    si.sku_description,
    si.sku_code,
    sq.uom,
    sq.gride_type,
    sq.box_calculation_uom as Box_calculation,

```

```

SUM(
CASE
WHEN dqwt.reference_type = 'sku_scan'
AND sp.season = 'winter'
THEN dqwt.amount
ELSE 0
END
) AS Winter_Earning,

SUM(
CASE
WHEN dqwt.reference_type = 'sku_scan'
AND (sp.season = 'core' OR sp.season IS NULL)
THEN dqwt.amount
ELSE 0
END
) AS Core_Earning,

SUM(dqwt.amount) AS Balance,

SUM(
CASE
WHEN dqwt.reference_type = 'topup'
THEN dqwt.amount
ELSE 0
END
) AS Topup_Earning,

ABS(SUM(
CASE
WHEN dqwt.reference_type = 'credit_note'
THEN dqwt.amount
ELSE 0
END
)) AS Credit_Note

FROM deactivated_qr_wallet_transactions dqwt
JOIN users u
    ON u.id = dqwt.user_id

LEFT JOIN sku_inventories si

```

```

        ON si.id = dqwt.reference_id AND dqwt.reference_type = 'sku_scan'

LEFT JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sq
    ON si.sku_code = sq.sku_code
    AND si.uom = sq.uom

WHERE dqwt.created_at < '2026-04-01'
AND u.user_role = 4

GROUP BY
    u.distributor_id,
    DATE(dqwt.created_at),
    si.lpn_number,
    si.sku_description,
    si.sku_code,
    sq.uom,
    sq.gride_type,
    sq.box_calculation_uom;

```

– Distributor Targets Report (Quarter Wise)

Select * FROM companies ;

– To check Redeemtion

```

select user_id , sum(amount) from scheme_wallet_transactions
where scheme_id = 3
and reference_type in ('scheme_gift_order','cash_convert')
group by user_id ;

```

– To check Balance and Earnings

```

SELECT
    user_id,
    SUM(CASE
        WHEN reference_type = 'sku_scan'
        THEN amount
        ELSE 0
    )

```

```
        END) AS total_earning,  
        SUM(amount) AS total_balance  
FROM scheme_wallet_transactions  
GROUP BY user_id;
```

-- bonus Balance and Cash Balance

```
SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END)  
-  
SUM(CASE WHEN type = 'Debit' THEN points ELSE 0 END)  
AS bonus_balance
```

-- for retailers

```
SELECT  
    u.id,  
    u.name,  
    u.mobile_number,  
    COALESCE(w.cash_balance, 0) AS Cash_Balance,  
  
    COALESCE(b.bonus_balance, 0) AS Bonus_Balance
```

```
FROM users u
```

```
LEFT JOIN (  
    SELECT  
        user_id,  
        SUM(amount) AS cash_balance  
    FROM wallet_transaction  
    WHERE DATE(created_at) <= '2026-04-30'  
    GROUP BY user_id  
) w ON u.id = w.user_id
```

```
LEFT JOIN (  
    SELECT  
        user_id,  
        SUM(CASE  
            WHEN type = 'credit' THEN points  
            WHEN type = 'Debit' THEN -points  
            ELSE 0
```

```

        END) AS bonus_balance
    FROM bonus_transactions
    GROUP BY user_id
) b ON u.id = b.user_id

```

```

WHERE u.user_role = 2;

```

-- For Wholesaler only cash balance

```

SELECT
    u.id,
    u.name,
    u.mobile_number,
    COALESCE(w.cash_balance, 0) AS Cash_Balance,

```

```

FROM users u

```

```

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS cash_balance
    FROM wallet_transaction
    WHERE DATE(created_at) <= '2026-04-30'
    GROUP BY user_id
) w ON u.id = w.user_id

```

```

WHERE u.user_role = 5;

```

-- retaier scan mapped / not - checking!!

```

SELECT * FROM sku_inventories WHERE lpn_number IN
('UASVMAQ','FULNVGE','OGAZYIG','HHBKBGL','CNKVRIL'); -- here take sku_code
SELECT * FROM sku_qr_points_maps where sku_code = "ACM001BRN2PC#90";

```

-- Growth Scheme (unit scanned , Points, bonus , growth, total earned points)

```

SELECT
    u.id,
    u.state_id,
    u.city,
    u.user_role,

```

```

u.pincode,
m.monthly_average,

COALESCE(us.Unit_Scanned, 0) AS Unit_Scanned,
COALESCE(mp.Points, 0) AS Points,
COALESCE(bp.Bonus_Points, 0) AS Bonus_Points,
COALESCE(gp.Growth_Points, 0) AS Growth_Points,
(
COALESCE(bp.Bonus_Points, 0) +
COALESCE(gp.Growth_Points, 0)
) AS Total_Points_Earned

FROM users u

JOIN monthly_averages m
    ON u.id = m.user_id

-- Unit Scanned
LEFT JOIN (
    SELECT
        si.status_retailer_id ,
        SUM(sq.box_calculation_uom) AS Unit_Scanned
    FROM sku_inventories si
    JOIN sku_qr_points_maps sq
    ON si.uom = sq.uom
    AND si.sku_code = sq.sku_code
    GROUP BY si.status_retailer_id
) us
    ON u.id = us.status_retailer_id

-- Total Points
LEFT JOIN (
    SELECT
        users_id,
        SUM(points) AS Points
    FROM mypoints
    WHERE created_at >= '2026-03-01'
    AND created_at < '2026-04-01'
    GROUP BY users_id
) mp
    ON u.id = mp.users_id

```

```

-- Bonus Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Bonus_Points
    FROM wallet_transaction
    WHERE reference_type = 'topup'
    AND reference_id REGEXP '^(13|19|20|21|22)'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) bp
    ON u.id = bp.user_id

```

```

-- Growth Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Growth_Points
    FROM wallet_transaction
    WHERE reference_type = 'incentive'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) gp
    ON u.id = gp.user_id

```

```

WHERE u.user_role = 2;

```

– Toing_scheme_karnataka

```

SELECT
    si.distributer_id,
    u.`name`,
    si.status_retailer_id,
    si.retailer_scanned_at,
    sp.product_name,
    sp.product_code,

    -- Earned Units (only sku_scan)
    COALESCE(SUM(
        CASE

```

```

        WHEN wt.reference_type = 'sku_scan'
        THEN wt.amount
        ELSE 0
    END
), 0) AS Earned_Units,
csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id

JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code
    AND sqpm.uom = si.uom

JOIN category_sub_groups csg
    ON csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id
    AND wt.scheme_id = 3 -- keep scheme filter here

WHERE u.user_role = 2
AND u.id = 50547

GROUP BY
    si.distributer_id,
    u.name,
    si.status_retailer_id,
    si.retailer_scanned_at,
    sp.product_name,
    sp.product_code,
    csg.name;

```

– toing scheme karnataka with more columns

```

SELECT
    si.distributer_id,

```

```

u.`name`,
u.mobile_number,
si.status_retailer_id,
si.retailer_scanned_at,
sp.product_name,
sp.product_code,
si.lpn_number,
si.uom,
sqpm.gride_type,
SUM(sqpm.box_calculation_uom) AS Total_Box_Cal_UOM,

-- Earned Units (only sku_scan)
COALESCE(SUM(
    CASE
        WHEN wt.reference_type = 'sku_scan'
        THEN wt.amount
        ELSE 0
    END
), 0) AS Earned_Units,
csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id

JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code
    AND sqpm.uom = si.uom

JOIN category_sub_groups csg
    ON csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id
    AND wt.scheme_id = 3 -- keep scheme filter here

WHERE u.user_role = 2
AND u.id = 74505

```

GROUP BY

si.distributer_id,
u.name,
u.mobile_number,
si.status_retailer_id,
si.retailer_scanned_at,
sp.product_name,
sp.product_code,
si.uom,
csg.name,
si.lpn_number,
sqpm.gride_type;

-- Deactivated unscanned report (open balance , C.N - winter & core)

Balance (calc manually in sheet) = Open_winter_Balance + Open_core_Balance +
Winter_Earning + Core_Earning + Topup_Earning - Credit_Note_winter + Credit_Note_core (C N)

SELECT

t.*,

-- Core Balance

(
t.Open_core_Balance
+ t.Core_Earning
+ t.Topup_Earning
- t.Credit_Note_core
) AS Core_Balance,

-- Winter Balance

(
t.Open_winter_Balance
+ t.Winter_Earning
+ t.Topup_Earning
- t.Credit_Note_winter
) AS Winter_Balance

FROM

(

SELECT

u.distributer_id,

-- Opening Balance (Before April 1)

SUM(

CASE

WHEN dqwt.created_at < '2026-04-01'

THEN dqwt.amount

ELSE 0

END

) AS Open_Balance,

-- Opening Winter Balance

SUM(

CASE

WHEN dqwt.created_at < '2026-04-01'

AND sp.season = 'winter'

THEN dqwt.amount

ELSE 0

END

) AS Open_winter_Balance,

-- Opening Core Balance

SUM(

CASE

WHEN dqwt.created_at < '2026-04-01'

AND (sp.season = 'core' OR sp.season IS NULL)

THEN dqwt.amount

ELSE 0

END

) AS Open_core_Balance,

-- Winter Earnings (April - June)

SUM(

CASE

WHEN dqwt.created_at >= '2026-04-01'

AND dqwt.created_at < '2026-07-01'

AND dqwt.reference_type = 'sku_scan'

AND sp.season = 'winter'

THEN dqwt.amount

ELSE 0

END

```
) AS Winter_Earning,
```

```
-- Core Earnings (April - June)
```

```
SUM(  
  CASE  
    WHEN dqwt.created_at >= '2026-04-01'  
    AND dqwt.created_at < '2026-07-01'  
    AND dqwt.reference_type = 'sku_scan'  
    AND (sp.season = 'core' OR sp.season IS NULL)  
    THEN dqwt.amount  
    ELSE 0  
  END  
) AS Core_Earning,
```

```
-- Topup Earnings (April - June)
```

```
SUM(  
  CASE  
    WHEN dqwt.created_at >= '2026-04-01'  
    AND dqwt.created_at < '2026-07-01'  
    AND dqwt.reference_type = 'topup'  
    THEN dqwt.amount  
    ELSE 0  
  END  
) AS Topup_Earning,
```

```
-- Total Credit Note
```

```
ABS(  
  SUM(  
    CASE  
      WHEN dqwt.created_at >= '2026-04-01'  
      AND dqwt.created_at < '2026-07-01'  
      AND dqwt.reference_type = 'credit_note'  
      THEN dqwt.amount  
      ELSE 0  
    END  
  )  
)
```

```
) AS Credit_Note,
```

```
-- Winter Credit Note
```

```
ABS(  
  SUM(  
    CASE
```

```

        WHEN dqwt.created_at >= '2026-04-01'
        AND dqwt.created_at < '2026-07-01'
        AND dqwt.reference_type = 'credit_note'
        AND dqwt.season = 'winter'
        THEN dqwt.amount
        ELSE 0
    END
)
) AS Credit_Note_winter,

```

-- Core Credit Note

```

ABS(
    SUM(
        CASE
            WHEN dqwt.created_at >= '2026-04-01'
            AND dqwt.created_at < '2026-07-01'
            AND dqwt.reference_type = 'credit_note'
            AND dqwt.season = 'core'
            THEN dqwt.amount
            ELSE 0
        END
    )
) AS Credit_Note_core

```

```

FROM deactivated_qr_wallet_transactions dqwt
JOIN users u
    ON u.id = dqwt.user_id
LEFT JOIN sku_inventories si
    ON si.id = dqwt.reference_id
LEFT JOIN sku_products sp
    ON sp.id = si.product_id
WHERE u.user_role = 4
GROUP BY u.distributer_id
) AS t;

```

-- Retailers Scan_count(UOM) & Cash_point

```

select user_id , sum(amount), COUNT(created_at) from wallet_transaction
where user_id in (

)

```

```
and reference_type IN ('cash_point','topup')
and created_at >= '2026-06-01'
and created_at < '2026-07-01'
group by user_id;
```

-- Retailer scan unit point and scan count (not cash point)

```
select si.status_retailer_id,sum(sq.box_calculation_uom),count(si.retailer_scanned_at)
FROM sku_inventories si
JOIN sku_qr_points_maps sq
  ON si.sku_code = sq.sku_code and si.uom = sq.uom
where si.retailer_scanned_at >= '2026-06-01'
  AND si.retailer_scanned_at < '2026-07-01'
AND si.status_retailer_id in (74417) group by si.status_retailer_id;
```

-- retailers list who has not scanned for last 2 months

```
SELECT
  u.id,
  u.name,
  u.city,
  u.pincode,
  u.address,
  u.email,
  u.mobile_number
FROM users u
WHERE u.state_id = 8
  AND u.user_role = 2
  AND NOT EXISTS (
    SELECT 1
    FROM sku_inventories si
    WHERE si.status_retailer_id = u.id
      AND si.retailer_scanned_at >= '2026-05-01'
      AND si.retailer_scanned_at < '2026-07-01'
  );
```

-- New Retailers Onboarded list for May Month

```
WITH distributor_base AS (  
  SELECT DISTINCT distributor_id  
  FROM executive_distributor_mappings  
  WHERE executive_id = 53478  
)
```

```
new_retailers AS (  
  SELECT  
    u.id AS retailer_id,  
    md.shop_name,  
    u.mobile_number,  
    u.mobile_number AS whatsapp_number,  
    u.city,  
    d.district_name,  
    u.created_at AS onboarded_date  
  FROM users u  
  LEFT JOIN mechanic_details md  
    ON md.mechanic_id = u.id  
  LEFT JOIN pincode_address_maps pam  
    ON pam.pincode = u.pincode  
  LEFT JOIN districts d  
    ON d.id = pam.district_id  
  LEFT JOIN sku_inventories si  
    ON si.status_retailer_id = u.id  
  WHERE u.primary_distributor_id IN (  
    SELECT distributor_id  
    FROM distributor_base  
  )  
  AND u.user_role = 2  
  AND u.status = 'approved'  
  AND u.created_at >= '2026-03-01'  
  AND u.created_at < '2026-04-01'  
  AND si.retailer_scanned_at >= '2026-03-01' – this throws duplicate here  
  AND si.retailer_scanned_at < '2026-04-01'  
)
```

```
scan_data AS (  
  SELECT  
    si.status_retailer_id AS retailer_id,  
    si.sku_code,  
    si.uom,  
    CASE
```

```

        WHEN sqm.box_calculation_uom IS NOT NULL
            AND sqm.box_calculation_uom > 0
            THEN sqm.box_calculation_uom
        WHEN si.uom = 'B5'
            THEN 0.5
        ELSE 1
    END AS box_value
FROM sku_inventories si
LEFT JOIN sku_qr_points_maps sqm
    ON sqm.sku_code = si.sku_code
    AND sqm.uom = si.uom
WHERE si.retailer_scanned_at >= '2026-03-01'
    AND si.retailer_scanned_at < '2026-04-01'
),

retailer_boxes AS (
    SELECT
        retailer_id,
        COUNT(*) AS scan_count,
        SUM(box_value) AS scanned_boxes
    FROM scan_data
    GROUP BY retailer_id
)

SELECT
    nr.retailer_id,
    nr.shop_name,
    nr.mobile_number,
    nr.whatsapp_number,
    nr.city,
    nr.district_name,
    DATE(nr.onboarded_date) AS onboarded_date,
    COALESCE(rb.scan_count, 0) AS scan_count,
    COALESCE(rb.scanned_boxes, 0) AS scanned_boxes
FROM new_retailers nr
LEFT JOIN retailer_boxes rb
    ON rb.retailer_id = nr.retailer_id
ORDER BY nr.onboarded_date;

```

-- bonus where remark = only bonus then use wallet transaction

```
select wt.* FROM wallet_transaction wt join users u on u.id = wt.user_id
WHERE wt.reference_type in ('topup', 'incentive')
AND reference_id is not null
AND wt.created_at >= '2026-07-01 00:00:00'
AND wt.created_at < '2026-08-05 00:00:00'
and u.user_role = 2;
```

-- Monthly average for retailers

```
select m.* FROM monthly_averages m join users u on u.id = m.user_id
WHERE u.user_role = 2;
```

JHG Field Sales (php to SQL queries)

– Onboarding Targets

```
WITH quarter_target_cte AS (  
    SELECT  
        ot.se_id,  
        CASE  
            WHEN 3 BETWEEN 4 AND 6 THEN ot.q1_target  
            WHEN 3 BETWEEN 7 AND 9 THEN ot.q2_target  
            WHEN 3 BETWEEN 10 AND 12 THEN ot.q3_target  
            ELSE ot.q4_target  
        END AS quarter_target  
    FROM onboarding_targets ot  
    WHERE ot.se_id = 53479  
)
```

```

monthly_target_cte AS (
    SELECT
        COALESCE(quarter_target / 3, 0) AS monthly_target
    FROM quarter_target_cte
),

distributor_base AS (
    SELECT DISTINCT distributor_id
    FROM executive_distributor_mappings
    WHERE executive_id = 53479
),

new_retailers AS (
    SELECT
        u.id AS retailer_id,
        md.shop_name,
        u.mobile_number,
        u.mobile_number AS whatsapp_number,
        u.city,
        d.district_name
    FROM users u
    LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
    LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode
    LEFT JOIN districts d ON d.id = pam.district_id
    WHERE u.primary_distributor_id IN (SELECT distributor_id FROM distributor_base)
    AND u.user_role = 2
    AND u.status = 'approved'
    AND u.created_at BETWEEN '2026-03-01 00:00:00' AND '2026-03-31 23:59:59'
),

scan_data AS (
    SELECT
        si.status_retailer_id AS retailer_id,
        si.uom,
        CASE
            WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0
            THEN sqm.box_calculation_uom
            WHEN si.uom = 'B5'
            THEN 0.5
            ELSE 1
        END AS box_value
    FROM sku_inventories si
    LEFT JOIN sku_qr_points_maps sqm

```

```

        ON si.sku_code = sqm.sku_code
        AND si.uom = sqm.uom
        WHERE si.retailer_scanned_at BETWEEN '2026-03-01 00:00:00'
            AND '2026-03-31 23:59:59'
    ),

    retailer_boxes AS (
        SELECT
            nr.retailer_id,
            nr.shop_name,
            nr.mobile_number,
            nr.whatsapp_number,
            nr.city,
            nr.district_name,
            COALESCE(SUM(sd.box_value), 0) AS scanned_boxes
        FROM new_retailers nr
        LEFT JOIN scan_data sd
            ON sd.retailer_id = nr.retailer_id
        GROUP BY
            nr.retailer_id,
            nr.shop_name,
            nr.mobile_number,
            nr.whatsapp_number,
            nr.city,
            nr.district_name
    ),

    qualified_retailers AS (
        SELECT *
        FROM retailer_boxes
        WHERE scanned_boxes >= 10
    )
SELECT
    (SELECT monthly_target FROM monthly_target_cte LIMIT 1) AS monthly_target,

    -- Monthly achievement
    (SELECT COUNT(*) FROM new_retailers) AS monthly_achievement,

    -- All retailers
    (SELECT COUNT(*) FROM retailer_boxes) AS total_new_retailers,
    (SELECT COALESCE(SUM(scanned_boxes),0) FROM retailer_boxes) AS total_boxes_all,

    -- Qualified retailers

```

```
(SELECT COUNT(*) FROM qualified_retailers) AS qualified_retailers_count,  
(SELECT COALESCE(SUM(scanned_boxes),0) FROM qualified_retailers) AS qualified_total_boxes;
```

-- new category

WITH TargetRetailers AS (

-- 1. Get the Executive Auth and their associated Approved Retailers

```
SELECT u.id AS retailer_id  
FROM users u  
WHERE u.primary_distributor_id IN (  
SELECT distributor_id  
FROM executive_distributor_mappings  
WHERE executive_id = 53479  
)  
AND u.user_role = 2  
AND u.status = 'approved'
```

),

PrevData AS (

-- 2. Aggregate Previous Data: From Financial Year Start (April 1st) string calculated dynamically

-> until Start of Current Month

```
SELECT  
si.status_retailer_id as retailer_id,  
sp.categories as category_id,  
SUM(  
CASE  
WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN  
sqm.box_calculation_uom  
WHEN si.uom = 'B5' THEN 0.5  
ELSE 1  
END  
) as prev_total_boxes  
FROM sku_inventories si  
JOIN sku_products sp ON sp.id = si.product_id  
LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom  
JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id  
JOIN users u ON u.id = si.status_retailer_id  
WHERE u.created_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00') -- Created before this  
month  
-- FY Date Logic mimicking PHP $fyStartDate  
AND si.retailer_scanned_at >= STR_TO_DATE(CONCAT(CASE WHEN MONTH(NOW()) >= 4 THEN  
YEAR(NOW()) ELSE YEAR(NOW()) - 1 END, '-04-01'), '%Y-%m-%d')  
-- Up to the end of last month
```

```

AND si.retailer_scanned_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
GROUP BY si.status_retailer_id, sp.categories
),
CurrentData AS (
    -- 3. Aggregate Current Data (This month till date)
    SELECT
        si.status_retailer_id as retailer_id,
        sp.categories as category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) as current_total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
    JOIN users u ON u.id = si.status_retailer_id
    WHERE u.created_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
    -- From start of this month till right NOW
    AND si.retailer_scanned_at >= DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
    AND si.retailer_scanned_at <= NOW()
    GROUP BY si.status_retailer_id, sp.categories
),
NewCategoryActivations AS (
    -- 4. Apply The Business Logic (Current Data where Previous Data was < 5 or NULL)
    SELECT
        c.retailer_id,
        c.category_id,
        c.current_total_boxes,
        mc.name as category_name,
        CASE WHEN c.current_total_boxes >= 5 THEN 1 ELSE 0 END as is_qualified
    FROM CurrentData c
    LEFT JOIN PrevData p ON c.retailer_id = p.retailer_id AND c.category_id = p.category_id
    JOIN master_categories mc ON mc.id = c.category_id
    WHERE (p.prev_total_boxes IS NULL OR p.prev_total_boxes < 5)
)
-- 5. Final Output joined with Retailer profiles
SELECT

```

```

        nca.retailer_id,
        md.shop_name,
        u.mobile_number,
        u.mobile_number as whatsapp_number,
        u.city,
        nca.category_id,
        nca.category_name,
        nca.current_total_boxes as boxes_scanned,
        nca.is_qualified
FROM NewCategoryActivations nca
JOIN users u ON u.id = nca.retailer_id
LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
ORDER BY nca.retailer_id, nca.category_id;

```

-- incremental sales

```

WITH distributor_base AS (
    SELECT distributor_id
    FROM executive_distributor_mappings
    WHERE executive_id = 53479
),

/* -----
   DATE CONSTANTS
   -----*/
date_bounds AS (
    SELECT
        '2026-02-01 00:00:00' AS start_date,
        '2026-02-28 23:59:59' AS end_date,
        '2025-04-01 00:00:00' AS fy_start,
        '2026-01-31 23:59:59' AS last_month_end
),

/* -----
   NEW RETAILERS (MARCH)
   -----*/
new_retailers AS (
    SELECT id
    FROM users
    WHERE primary_distributor_id IN (SELECT distributor_id FROM distributor_base)
    AND user_role = 2
    AND status = 'approved'

```

```

        AND created_at BETWEEN (SELECT start_date FROM date_bounds)
        AND (SELECT end_date FROM date_bounds)
    ),

/* -----
CURRENT MONTH DATA (NEW RETAILERS)
-----*/
current_new_data AS (
    SELECT
        si.status_retailer_id AS retailer_id,
        sp.categories AS category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) AS total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm
    ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    WHERE si.status_retailer_id IN (SELECT id FROM new_retailers)
    AND si.retailer_scanned_at BETWEEN
    (SELECT start_date FROM date_bounds)
    AND (SELECT end_date FROM date_bounds)
    GROUP BY si.status_retailer_id, sp.categories
),

/* -----
QUALIFIED RETAILERS (>=10 BOXES)
-----*/
qualified_retailers AS (
    SELECT retailer_id, SUM(total_boxes) AS total_boxes
    FROM current_new_data
    GROUP BY retailer_id
    HAVING SUM(total_boxes) >= 10
),

qualified_total AS (
    SELECT SUM(total_boxes) AS total_boxes
    FROM qualified_retailers

```

),

/* -----

OLD RETAILERS

-----*/

old_retailers AS (

SELECT id

FROM users

WHERE primary_distributor_id IN (SELECT distributor_id FROM distributor_base)

AND user_role = 2

AND status = 'approved'

AND created_at < (SELECT start_date FROM date_bounds)

),

/* -----

PREVIOUS CATEGORY DATA

-----*/

prev_category_data AS (

SELECT

si.status_retailer_id AS retailer_id,

sp.categories AS category_id,

SUM(

CASE

WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom

WHEN si.uom = 'B5' THEN 0.5

ELSE 1

END

) AS total_boxes

FROM sku_inventories si

JOIN sku_products sp ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sqm

ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom

WHERE si.status_retailer_id IN (SELECT id FROM old_retailers)

AND si.retailer_scanned_at BETWEEN

(SELECT fy_start FROM date_bounds)

AND (SELECT last_month_end FROM date_bounds)

GROUP BY si.status_retailer_id, sp.categories

),

/* -----

CURRENT CATEGORY DATA

-----*/

```

current_category_data AS (
    SELECT
        si.status_retailer_id AS retailer_id,
        sp.categories AS category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) AS total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm
    ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    WHERE si.status_retailer_id IN (SELECT id FROM old_retailers)
    AND si.retailer_scanned_at BETWEEN
    (SELECT start_date FROM date_bounds)
    AND (SELECT end_date FROM date_bounds)
    GROUP BY si.status_retailer_id, sp.categories
),

/* -----
NEW CATEGORY BOXES
-----*/
new_category_boxes AS (
    SELECT SUM(c.total_boxes) AS total_boxes
    FROM current_category_data c
    LEFT JOIN prev_category_data p
    ON c.retailer_id = p.retailer_id
    AND c.category_id = p.category_id
    WHERE
    (p.total_boxes IS NULL AND c.total_boxes >= 5)
    OR (p.total_boxes < 5)
)

/* -----
FINAL OUTPUT
-----*/
SELECT
    (SELECT total_boxes FROM qualified_total) +
    (SELECT total_boxes FROM new_category_boxes) AS total_incremental_sales,

```

```
(SELECT total_boxes FROM qualified_total)
AS total_boxes_by_qualified_retailers,
```

```
(SELECT total_boxes FROM new_category_boxes)
AS total_box_scanned_from_new_category;
```

-- new category april1 - april 17

```
WITH TargetRetailers AS (
    SELECT u.id AS retailer_id
    FROM users u
    WHERE u.primary_distributor_id IN (
        SELECT distributor_id
        FROM executive_distributor_mappings
        WHERE executive_id = 53509
    )
    AND u.user_role = 2
    AND u.status = 'approved'
),
```

```
PrevData AS (
    -- From FY start (April 1, 2026) → before April 1, 2026 (no change in logic intent)
    SELECT
        si.status_retailer_id as retailer_id,
        sp.categories as category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) as prev_total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
    JOIN users u ON u.id = si.status_retailer_id
    WHERE u.created_at < '2026-04-01 00:00:00'
    AND si.retailer_scanned_at >= '2026-04-01 00:00:00'
```

```

AND si.retailer_scanned_at < '2026-04-01 00:00:00'
GROUP BY si.status_retailer_id, sp.categories
),

CurrentData AS (
    -- April 1 → April 17, 2026
    SELECT
        si.status_retailer_id as retailer_id,
        sp.categories as category_id,
SUM(
    CASE
        WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
sqm.box_calculation_uom
        WHEN si.uom = 'B5' THEN 0.5
        ELSE 1
    END
) as current_total_boxes
FROM sku_inventories si
JOIN sku_products sp ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
JOIN users u ON u.id = si.status_retailer_id
WHERE u.created_at < '2026-04-01 00:00:00'
AND si.retailer_scanned_at >= '2026-04-01 00:00:00'
AND si.retailer_scanned_at <= '2026-04-17 23:59:59'
GROUP BY si.status_retailer_id, sp.categories
),

NewCategoryActivations AS (
    SELECT
        c.retailer_id,
        c.category_id,
        c.current_total_boxes,
        mc.name as category_name,
        CASE WHEN c.current_total_boxes >= 5 THEN 1 ELSE 0 END as is_qualified
    FROM CurrentData c
    LEFT JOIN PrevData p
    ON c.retailer_id = p.retailer_id
    AND c.category_id = p.category_id
    JOIN master_categories mc ON mc.id = c.category_id
    WHERE (p.prev_total_boxes IS NULL OR p.prev_total_boxes < 5)
)
SELECT

```

```

        nca.retailer_id,
        md.shop_name,
        u.mobile_number,
        u.mobile_number as whatsapp_number,
        u.city,
        nca.category_id,
        nca.category_name,
        nca.current_total_boxes as boxes_scanned,
        nca.is_qualified
FROM NewCategoryActivations nca
JOIN users u ON u.id = nca.retailer_id
LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
ORDER BY nca.retailer_id, nca.category_id;

```

#To Check Earning for Retailers

```

select sum(amount)
from wallet_transaction where reference_type IN ('cash_point','topup','referral_earning','incentive')
and user_id = 67417 and created_at BETWEEN '2025-03-01' and '2026-03-31';

```

#To Check cash redeemed for Retailers

```

select sum(amount)
from wallet_transaction where reference_type IN ('withdrawal','bonus_conversion')
and user_id = 67417 and created_at BETWEEN '2025-03-01' and '2026-03-31';

```

#To Check gift redeemed for Retailers

```

select sum(used_points)
from gift_orders
where status in ('Pending','Delivered')
and user_id = 67417;

```

#To Check TDS for Retailers

```

select * from withdrawal_request where user_id = 53756;

```

#To Check total_bonus for Retailers

```

SELECT SUM(points)

```

```
FROM bonus_transactions
WHERE type = 'credit' and reference_type is null
AND user_id = 67417
AND created_at BETWEEN '2025-03-01' AND '2026-03-31';
```

#To Check Total Balance for retailers

```
select sum(amount) from wallet_transaction where user_id = 53756 and created_at BETWEEN
'2025-06-01' AND '2026-03-31';
```

```
select sum(amount) from wallet_transaction where user_id = 53756;
```

```
SELECT SUM(amount)
FROM wallet_transaction
WHERE user_id = 53756
AND created_at >= '2025-06-01';
```

#To Check Bonus Balance for retailers

```
select sum(points) from bonus_transactions where user_id = 53756 and type = 'credit';
select sum(points) from bonus_transactions where user_id = 53756 and type = 'debit';
```

#Karnataka Toing Scheme 2026

```
SELECT
    si.distributer_id,
    si.id,
    u.`name`,
    si.sku_code,
    si.lpn_number,
    si.uom,
    si.status_retailer_id,
    si.retailer_scanned_at,
    si.created_at,
```

```

        'Retailer' as role_name,
        sqpm.gride_type,
        sqpm.box_calculation_uom,    sp.product_name,
        COALESCE(wt.amount, 0) AS Earned_Units,
        sqpm.pcs_calculation_uom,
        csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id
JOIN sku_products sp
    ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code and sqpm.uom = si.uom
join category_sub_groups csg on
    csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id and wt.reference_type = 'sku_scan'    and scheme_id = 3
WHERE u.user_role = 2;

```

#Distributor Daily Scanning Report

```

WITH user_info AS (
    SELECT u.id,
        COALESCE(m.shop_name, u.name) AS customer_name,
        u.pincode, u.city,
        pam.city AS actual_city,
        s.sname AS state,
        z.name AS zone,
        u.user_role
    FROM users u
    LEFT JOIN mechanic_details m ON m.mechanic_id = u.id

```

```

LEFT JOIN state s ON s.id = u.state_id
LEFT JOIN zones z ON z.id = s.zone_id
LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode
WHERE u.user_role IN (2,5)
),

box_scans AS (
  SELECT reference_id AS skui_id, schemes.scheme_name,
         swt.amount AS earned_units, 'Scheme Scan' AS scan_type
  FROM scheme_wallet_transactions swt
  JOIN schemes ON schemes.id = swt.scheme_id
  WHERE swt.reference_type = 'sku_scan'
        AND swt.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'

  UNION ALL

  SELECT reference_id, NULL, gwt.amount, 'Gift Scan'
  FROM gift_scan_wallet_transactions gwt
  WHERE gwt.reference_type = 'sku_scan'
        AND gwt.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'

  UNION ALL

  SELECT reference_id, NULL, wst.amount, 'Winter Scan'
  FROM winter_scan_wallet_transactions wst
  WHERE wst.reference_type = 'sku_scan'
        AND wst.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'
),

scan_data AS (
  SELECT skui.id,
         skui.lpn_number,
         COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
         sqpm.box_calculation_uom,
         sqpm.pcs_calculation_uom,
         DATE(skui.created_at) AS created_at,
         skui.sku_code, skui.sku_description,
         p.product_name, p.product_code,
         p.season AS product_type,
         mc.name AS category_name,
         cg.name AS category_sub_group,
         b.brand_name AS style,

```

```

d.name AS distributor,
d.sap_code,
dis_st.sname AS distributor_state,
DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
DATE(skui.retailer_scanned_at) AS retailer_scanned_on,

CASE
    WHEN WEEK(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at),0)
        - WEEK(
            DATE_SUB(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at),
                INTERVAL
DAYOFMONTH(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at))-1 DAY),0)+1 = 1 THEN
'Week 1'
        WHEN WEEK(...) = 2 THEN 'Week 2'
        WHEN WEEK(...) = 3 THEN 'Week 3'
        WHEN WEEK(...) = 4 THEN 'Week 4'
        ELSE 'Week 5'
    END AS scan_week,

CASE
    WHEN skui.status_retailer_id IS NOT NULL AND skui.status_wholeseller_id IS NULL THEN
'Retailer'
    WHEN skui.status_wholeseller_id IS NOT NULL AND skui.status_retailer_id IS NULL THEN
'Wholesaler'
    ELSE 'Both'
    END AS scan_by,

COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,
CASE WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer' ELSE 'Wholesaler' END AS
customer_type

FROM sku_inventories skui
JOIN sku_products p ON p.id = skui.product_id
LEFT JOIN master_categories mc ON mc.id = p.categories
LEFT JOIN category_sub_groups cg ON cg.id = p.category_sub_group_id
LEFT JOIN brands b ON b.id = p.brand
JOIN companies d ON d.id = skui.distributer_id
JOIN users du ON du.distributer_id = d.id AND du.user_role = 4
LEFT JOIN state dis_st ON dis_st.id = du.state_id
LEFT JOIN (
    SELECT sku_code, uom,
        MIN(box_calculation_uom) AS box_calculation_uom,
        MIN(pcs_calculation_uom) AS pcs_calculation_uom

```

```

        FROM sku_qr_points_maps
        GROUP BY sku_code, uom
    ) sqpm
    ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')
    AND sqpm.uom = skui.uom

WHERE (skui.retailer_scanned_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59')
      OR (skui.wholesaler_scanned_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59')
)

SELECT sd.*, ui.customer_name, ui.city, ui.pincode, ui.state, ui.zone, ui.actual_city,
       COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,
       bs.scheme_name,
       bs.earned_units,
       mp.points AS earned_point

FROM scan_data sd
LEFT JOIN user_info ui ON ui.id = sd.customer_id
LEFT JOIN box_scans bs ON bs.skui_id = sd.id
LEFT JOIN mypoints mp ON mp.sku_inventory_id = sd.id
                     AND mp.users_id = sd.customer_id

WHERE sd.sap_code = 'D12IND2101';

```

```

WITH user_info AS (
    SELECT
        u.id,
        COALESCE(m.shop_name, u.name) AS customer_name,
        u.pincode,
        u.city,
        pam.city AS actual_city,
        s.sname AS state,
        z.name AS zone,
        u.user_role
    FROM users u
    LEFT JOIN mechanic_details m
        ON m.mechanic_id = u.id

```

```
LEFT JOIN state s
  ON s.id = u.state_id
LEFT JOIN zones z
  ON z.id = s.zone_id
LEFT JOIN pincode_address_maps pam
  ON pam.pincode = u.pincode
WHERE u.user_role IN (2,5)
),
```

```
box_scans AS (
  SELECT
    reference_id AS skui_id,
    schemes.scheme_name,
    swt.amount AS earned_units,
    'Scheme Scan' AS scan_type
  FROM scheme_wallet_transactions swt
  JOIN schemes
    ON schemes.id = swt.scheme_id
  WHERE swt.reference_type = 'sku_scan'
    AND swt.created_at >= '2025-02-01'
    AND swt.created_at < '2025-09-01'
```

UNION ALL

```
SELECT
  reference_id,
  NULL,
  gwt.amount,
  'Gift Scan'
FROM gift_scan_wallet_transactions gwt
WHERE gwt.reference_type = 'sku_scan'
  AND gwt.created_at >= '2025-02-01'
  AND gwt.created_at < '2025-09-01'
```

UNION ALL

```
SELECT
  reference_id,
  NULL,
  wst.amount,
  'Winter Scan'
FROM winter_scan_wallet_transactions wst
```

```

WHERE wst.reference_type = 'sku_scan'
AND wst.created_at >= '2025-02-01'
AND wst.created_at < '2025-09-01'
),

scan_data AS (
  SELECT
    skui.id,
    skui.lpn_number,

    COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
    sqpm.box_calculation_uom,
    sqpm.pcs_calculation_uom,

    DATE(skui.created_at) AS created_at,

    skui.sku_code,
    skui.sku_description,

    p.product_name,
    p.product_code,
    p.season AS product_type,

    mc.name AS category_name,
    cg.name AS category_sub_group,
    b.brand_name AS style,

    d.name AS distributor,
    d.sap_code,
    dis_st.sname AS distributor_state,

    DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
    DATE(skui.retailer_scanned_at) AS retailer_scanned_on,

    --  Week Number (clean)
    (
      WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
      - WEEK(
        DATE_SUB(
          COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
          INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at)) -
1 DAY

```

```

        ), 0
    ) + 1
) AS week_number,

CASE
    WHEN skui.status_retailer_id IS NOT NULL
        AND skui.status_wholeseller_id IS NULL THEN 'Retailer'
    WHEN skui.status_wholeseller_id IS NOT NULL
        AND skui.status_retailer_id IS NULL THEN 'Wholesaler'
    ELSE 'Both'
END AS scan_by,

COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,

CASE
    WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer'
    ELSE 'Wholesaler'
END AS customer_type

FROM sku_inventories skui

JOIN sku_products p
    ON p.id = skui.product_id

LEFT JOIN master_categories mc
    ON mc.id = p.categories

LEFT JOIN category_sub_groups cg
    ON cg.id = p.category_sub_group_id

LEFT JOIN brands b
    ON b.id = p.brand

JOIN companies d
    ON d.id = skui.distributer_id

--  fixed join (no data loss)
LEFT JOIN users du
    ON du.distributer_id = d.id
    AND du.user_role = 4

LEFT JOIN state dis_st

```

ON dis_st.id = du.state_id

LEFT JOIN (

SELECT

sku_code,

uom,

MIN(box_calculation_uom) AS box_calculation_uom,

MIN(pcs_calculation_uom) AS pcs_calculation_uom

FROM sku_qr_points_maps

GROUP BY sku_code, uom

) sqpm

ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')

AND sqpm.uom = skui.uom

--  correct date filter

WHERE

COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at) >= '2025-02-01'

AND COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at) < '2025-09-01'

)

SELECT

sd.*,

--  final week label

CASE

WHEN sd.week_number = 1 THEN 'Week 1'

WHEN sd.week_number = 2 THEN 'Week 2'

WHEN sd.week_number = 3 THEN 'Week 3'

WHEN sd.week_number = 4 THEN 'Week 4'

ELSE 'Week 5'

END AS scan_week,

ui.customer_name,

ui.city,

ui.pincode,

ui.state,

ui.zone,

ui.actual_city,

COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,

bs.scheme_name,

bs.earned_units,

```

        mp.points AS earned_point

FROM scan_data sd

LEFT JOIN user_info ui
    ON ui.id = sd.customer_id

LEFT JOIN box_scans bs
    ON bs.skui_id = sd.id

LEFT JOIN mypoints mp
    ON mp.sku_inventory_id = sd.id
    AND mp.users_id = sd.customer_id

WHERE sd.sap_code = 'D12IND2101';

```

– Karnataka Toing Scheme

```

SELECT
    si.distributer_id,
    si.id,
    u.`name`,
    si.sku_code,
    si.lpn_number,
    si.uom,
    si.status_retailer_id,
    si.retailer_scanned_at,
    si.created_at,
    'Retailer' as role_name,
    sqpm.gride_type,
    sqpm.box_calculation_uom,  sp.product_name,
    COALESCE(wt.amount, 0) AS Earned_Units,
    sqpm.pcs_calculation_uom,
    csg.name

FROM sku_inventories si

JOIN users u

```

```

    ON u.id = si.status_retailer_id
JOIN sku_products sp
    ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code and sqpm.uom = si.uom
join category_sub_groups csg on
csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id and wt.reference_type = 'sku_scan' and scheme_id = 3
WHERE u.user_role = 2;

```

#Growth Scheme Report for bonus points

```

SELECT
    wt.user_id,
    SUM(wt.amount) AS Earning_points,

    u.name,
    u.mobile_number,

    s.sname AS state,
    md.shop_name

FROM wallet_transaction wt

JOIN users u
    ON u.id = wt.user_id

JOIN state s
    ON u.state_id = s.id

JOIN mechanic_details md
    ON md.mechanic_id = u.id

```

```
WHERE wt.created_at >= '2026-02-01'
AND wt.created_at < '2026-04-01'
AND wt.reference_type = 'incentive'
```

```
GROUP BY
  wt.user_id,
  u.name,
  u.mobile_number,
  s.sname,
  md.shop_name;
```

#Growth Scheme feb to mar all required columns

```
SELECT
  u.id,
  u.state_id,
  u.city,
  u.user_role,
  u.pincode,
  m.monthly_average,

  COALESCE(us.Unit_Scanned, 0) AS Unit_Scanned,
  COALESCE(mp.Points, 0) AS Points,
  COALESCE(bp.Bonus_Points, 0) AS Bonus_Points,
  COALESCE(gp.Growth_Points, 0) AS Growth_Points,

  (
    COALESCE(bp.Bonus_Points, 0) +
    COALESCE(gp.Growth_Points, 0)
  ) AS Total_Points_Earned

FROM users u

JOIN monthly_averages m
  ON u.id = m.user_id

-- Unit Scanned
LEFT JOIN (
```

```

SELECT
    si.status_retailer_id ,
    SUM(sq.box_calculation_uom) AS Unit_Scanned
FROM sku_inventories si
JOIN sku_qr_points_maps sq
    ON si.uom = sq.uom
    AND si.sku_code = sq.sku_code
GROUP BY si.status_retailer_id
) us
    ON u.id = us.status_retailer_id

```

-- Total Points

```

LEFT JOIN (
    SELECT
        users_id,
        SUM(points) AS Points
    FROM mypoints
    WHERE created_at >= '2026-03-01'
    AND created_at < '2026-04-01'
    GROUP BY users_id
) mp
    ON u.id = mp.users_id

```

-- Bonus Points

```

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Bonus_Points
    FROM wallet_transaction
    WHERE reference_type = 'topup'
    AND reference_id REGEXP '^(13|19|20|21|22)'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) bp
    ON u.id = bp.user_id

```

-- Growth Points

```

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Growth_Points

```

```
FROM wallet_transaction
WHERE reference_type = 'incentive'
AND created_at >= '2026-04-01'
AND created_at < '2026-05-01'
GROUP BY user_id
) gp
ON u.id = gp.user_id

WHERE u.user_role = 2;
```

#Toing_Scheme_Report

#Retailer_Month_wise_Ipn_number and Box_count

```
SELECT
    u.id,
    c.sap_code,
    md.shop_name,
    u.primary_distributor_id,
    u.pan_card,
    u.adhaar,
    u.mobile_number,
    u.email,
    s.sname AS State_name,
    u.district,
    u.city,
    u.pincode,

    /* Apr 2025 */
    COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-04' THEN si.lpn_number
    END) AS Apr_2025_lpn_count,
    SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-04'
        THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
    END)
        ELSE 0 END) AS Apr_2025_uom_count,

    /* May 2025 */
    COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-05' THEN si.lpn_number
    END) AS May_2025_lpn_count,
    SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-05'
        THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
    END)
        ELSE 0 END) AS May_2025_uom_count,

    /* Jun 2025 */
    COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-06' THEN si.lpn_number
    END) AS Jun_2025_lpn_count,
    SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-06'
        THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
    END)
        ELSE 0 END)
```

```

ELSE 0 END) AS Jun_2025_uom_count,

/* Jul 2025 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-07' THEN si.lpn_number
END) AS Jul_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-07'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Jul_2025_uom_count,

/* Aug 2025 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-08' THEN si.lpn_number
END) AS Aug_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-08'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Aug_2025_uom_count,

/* Sep 2025 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-09' THEN si.lpn_number
END) AS Sep_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-09'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Sep_2025_uom_count,

/* Oct 2025 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-10' THEN si.lpn_number
END) AS Oct_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-10'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Oct_2025_uom_count,

/* Nov 2025 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-11' THEN si.lpn_number
END) AS Nov_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-11'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Nov_2025_uom_count,

```

```
/* Dec 2025 */  
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-12' THEN si.lpn_number  
END) AS Dec_2025_lpn_count,  
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-12'  
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1  
END)  
ELSE 0 END) AS Dec_2025_uom_count,
```

```
/* Jan 2026 */  
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-01' THEN si.lpn_number  
END) AS Jan_2026_lpn_count,  
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-01'  
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1  
END)  
ELSE 0 END) AS Jan_2026_uom_count,
```

```
/* Feb 2026 */  
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-02' THEN si.lpn_number  
END) AS Feb_2026_lpn_count,  
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-02'  
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1  
END)  
ELSE 0 END) AS Feb_2026_uom_count,
```

```
/* Mar 2026 */  
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-03' THEN si.lpn_number  
END) AS Mar_2026_lpn_count,  
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-03'  
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1  
END)  
ELSE 0 END) AS Mar_2026_uom_count,
```

```
/* Apr 2026 */  
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-04' THEN si.lpn_number  
END) AS Apr_2026_lpn_count,  
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-04'  
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1  
END)  
ELSE 0 END) AS Apr_2026_uom_count,
```

```
/* May 2026 */
```

```

COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-05' THEN si.lpn_number
END) AS May_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-05'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS May_2026_uom_count,

/* Jun 2026 */
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-06' THEN si.lpn_number
END) AS Jun_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-06'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Jun_2026_uom_count,

MAX(si.retailer_scanned_at) AS last_scan_date

FROM users u

LEFT JOIN companies c
ON c.id = u.primary_distributor_id

LEFT JOIN sku_inventories si
ON si.status_retailer_id = u.id

LEFT JOIN (
SELECT
sku_code,
uom,
MIN(box_calculation_uom) AS box_calculation_uom
FROM sku_qr_points_maps
GROUP BY sku_code, uom
) sqpm
ON sqpm.sku_code = REPLACE(si.sku_code, ' ', '')
AND sqpm.uom = si.uom

JOIN mechanic_details md
ON md.mechanic_id = u.id

LEFT JOIN state s
ON s.id = u.state_id

```

```
WHERE u.user_role = 2
```

```
GROUP BY
```

```
u.id,  
c.sap_code,  
md.shop_name,  
u.primary_distributor_id,  
u.pan_card,  
u.adhaar,  
u.mobile_number,  
u.email,  
s.sname,  
u.district,  
u.city,  
u.pincode;
```

#LPN Dump Data

```
SELECT
```

```
lpn_number,  
sku_code,  
sku_description,  
carton_no,  
uom,  
invoice_number,  
invoice_date,  
warehouse,  
sources,  
retailer_scanned_at,  
wholesaler_scanned_at,  
sap_code,
```

```
CASE
  WHEN is_callback = 0 THEN 'Distributor Generator'
  WHEN is_callback = 1 THEN 'API Scan'
  WHEN is_callback = 2 THEN 'CSV Uploaded'
  ELSE 'Unknown'
END AS lpn_source,
```

```
CASE
  WHEN status_retailer_id IS NOT NULL THEN 'YES'
  ELSE 'NO'
END AS retailer_scan_status,
```

```
CASE
  WHEN wholesaler_scanned_at IS NOT NULL THEN 'YES'
  ELSE 'NO'
END AS wholesaler_scan_status,
```

```
CASE
  WHEN is_active = 0 THEN 'YES'
  ELSE 'NO'
END AS distributor_deactivated_status
```

```
FROM sku_inventories
join companies on
companies.id = sku_inventories.distributer_id;
```

#Retailer,Wholesaller,Distributor may month earning

```
SELECT
    u.id,
    r.name AS user_type,
    u.name as User_name,
    u.mobile_number,
    md.shop_name,

    SUM(
        CASE
            WHEN wt.created_at BETWEEN '2026-05-01' AND '2026-05-31'
            AND wt.reference_type IN ('withdrawal','bonus_conversion')
            THEN wt.amount
            ELSE 0
        END
    ) AS Redemption,

    SUM(
        CASE
            WHEN wt.created_at BETWEEN '2026-05-01' AND '2026-05-31'
            AND wt.reference_type IN ('cash_point','topup','referral_earning','incentive')
            THEN wt.amount
            ELSE 0
        END
    ) AS Total_Earning,

    SUM(
        CASE
            WHEN wt.created_at < '2026-05-01'
            THEN wt.amount
            ELSE 0
        END
    ) AS Open_Balance,
    sum(wt.amount) as Cash_Balance

FROM users u
JOIN wallet_transaction wt
    ON wt.user_id = u.id
```

```
JOIN role r
      ON r.id = u.user_role
```

```
join mechanic_details md on
md.mechanic_id = u.id
```

```
WHERE u.user_role IN (2,5,4)
```

```
GROUP BY
      wt.user_id,
      r.name,
      u.name,
      u.mobile_number,
      md.shop_name;
```

#Distributor Unscan monthly Core Balance report

```
SELECT
      u.distributer_id,
      dqwt.created_at,
      si.lpn_number,
      si.sku_description,
      si.sku_code,
      sp.product_code,
      sp.size,
      si.uom,
      sq.box_calculation_uom,
      sq.gride_type,
```

c.sap_code,

SUM(

CASE

WHEN dqwt.reference_type IN ('sku_scan')

AND sp.season = 'core'

THEN dqwt.amount

ELSE 0

END

) AS core_Earning,

SUM(dqwt.amount) AS Balance,

SUM(

CASE

WHEN dqwt.reference_type IN ('topup')

THEN dqwt.amount

ELSE 0

END

) AS Topup_Earning,

SUM(

CASE

WHEN dqwt.reference_type IN ('credit_note') -- abs

THEN dqwt.amount

ELSE 0

END

) AS credit_note

FROM deactivated_qr_wallet_transactions dqwt

JOIN users u

ON u.id = dqwt.user_id

JOIN sku_inventories si

ON si.id = dqwt.reference_id

JOIN sku_products sp

ON sp.id = si.product_id

join sku_qr_points_maps sq on

sq.sku_code = si.sku_code

```
join companies c on  
c.id = si.distributer_id
```

```
WHERE dqwt.created_at >= '2026-04-01' and dqwt.created_at < '2026-06-13'  
AND u.user_role = 4 and u.distributer_id = 6147 and dqwt.season = 'core'  
GROUP BY u.distributer_id,  
        dqwt.created_at,  
        si.lpn_number,  
        si.sku_description,  
        si.sku_code,  
        sp.product_code,  
        sp.size,  
        si.uom,  
        sq.box_calculation_uom,  
        sq.gride_type,  
        c.sap_code;
```

GIFT Order Redemption

```
SELECT  
    og.order_id,  
    og.user_id AS customer_id,  
    og.shop_name,  
    u.mobile_number,  
    og.status,  
    og.used_points,  
    og.created_at,  
    og.quantity,  
    gc.`name` as Gift_Name,  
    og.order_date,  
    s.sname as State_Name,  
    gc.gift_value,  
  
SUM(
```

```

CASE
    WHEN wt.created_at <= '2026-06-28' THEN wt.amount
    ELSE 0
END
) AS `Wallet Balance Before Redemption`,

SUM(
    CASE
        WHEN bt.created_at <= '2026-06-28' THEN bt.points
        ELSE 0
    END
) AS `Bonus Points Before Redemption`

FROM gift_orders og
JOIN users u
    ON u.id = og.user_id
JOIN wallet_transaction wt
    ON wt.user_id = og.user_id
JOIN bonus_transactions bt
    ON bt.user_id = og.user_id
join gift_cards gc on
gc.id = og.gift_card_id
join state s on
s.id = u.state_id
WHERE og.created_at BETWEEN '2026-06-29' AND '2026-07-02'
GROUP BY
    og.order_id,
    og.user_id,
    og.shop_name,
    u.mobile_number,
    og.status,
    og.used_points,
    og.created_at,
    og.quantity,
    gc.`name`,
    og.order_date,
    s.sname,
    gc.gift_value;

```

#Distributor wise Retailer Data

```
WITH user_info AS (  
    SELECT u.id,  
           COALESCE(m.shop_name, u.name) AS customer_name,  
           u.pincode, u.city,  
           pam.city AS actual_city,  
           s.sname AS state,  
           z.name AS zone,  
           u.user_role  
    FROM users u  
    LEFT JOIN mechanic_details m ON m.mechanic_id = u.id  
    LEFT JOIN state s ON s.id = u.state_id  
    LEFT JOIN zones z ON z.id = s.zone_id  
    LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode  
    WHERE u.user_role IN (2,5)  
)  
box_scans AS (  
    SELECT reference_id AS skui_id, schemes.scheme_name,  
           swt.amount AS earned_units, 'Scheme Scan' AS scan_type  
    FROM scheme_wallet_transactions swt  
    JOIN schemes ON schemes.id = swt.scheme_id  
    WHERE swt.reference_type = 'sku_scan'  
           AND swt.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'  
    UNION ALL  
    SELECT reference_id, NULL, gwt.amount, 'Gift Scan'  
    FROM gift_scan_wallet_transactions gwt  
    WHERE gwt.reference_type = 'sku_scan'  
           AND gwt.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'  
    UNION ALL  
    SELECT reference_id, NULL, wst.amount, 'Winter Scan'  
    FROM winter_scan_wallet_transactions wst  
    WHERE wst.reference_type = 'sku_scan'  
           AND wst.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'  
)
```

```

scan_data AS (
    SELECT skui.id,
           skui.lpn_number,
           COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
           sqpm.box_calculation_uom,
           sqpm.pcs_calculation_uom,
           DATE(skui.created_at) AS created_at,
           skui.sku_code, skui.sku_description,
           p.product_name, p.product_code,
           p.season AS product_type,
           mc.name AS category_name,
           cg.name AS category_sub_group,
           b.brand_name AS style,
           d.name AS distributor,
           d.sap_code,
           dis_st.sname AS distributor_state,
           DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
           DATE(skui.retailer_scanned_at) AS retailer_scanned_on,
           CASE
               WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                   - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                   INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                   - 1 DAY), 0) + 1 = 1 THEN 'Week 1'
               WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                   - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                   INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                   - 1 DAY), 0) + 1 = 2 THEN 'Week 2'
               WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                   - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                   INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                   - 1 DAY), 0) + 1 = 3 THEN 'Week 3'
               WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                   - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                   INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                   - 1 DAY), 0) + 1 = 4 THEN 'Week 4'
               ELSE 'Week 5'
           END AS scan_week,
           CASE
               WHEN skui.status_retailer_id IS NOT NULL AND skui.status_wholeseller_id IS NULL THEN
                   'Retailer'
               WHEN skui.status_wholeseller_id IS NOT NULL AND skui.status_retailer_id IS NULL THEN
                   'Wholesaler'
           END

```

```

        ELSE 'Both'
    END AS scan_by,
    COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,
    CASE WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer' ELSE 'Wholesaler' END AS
customer_type
FROM sku_inventories skui
JOIN sku_products p ON p.id = skui.product_id
LEFT JOIN master_categories mc ON mc.id = p.categories
LEFT JOIN category_sub_groups cg ON cg.id = p.category_sub_group_id
LEFT JOIN brands b ON b.id = p.brand
JOIN companies d ON d.id = skui.distributer_id
JOIN users du ON du.distributer_id = d.id AND du.user_role = 4
LEFT JOIN state dis_st ON dis_st.id = du.state_id
LEFT JOIN (
    SELECT sku_code, uom,
        MIN(box_calculation_uom) AS box_calculation_uom,
        MIN(pcs_calculation_uom) AS pcs_calculation_uom
    FROM sku_qr_points_maps
    GROUP BY sku_code, uom
) sqpm
    ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')
    AND sqpm.uom = skui.uom
WHERE (skui.retailer_scanned_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59')
)
SELECT sd.*, ui.customer_name, ui.city, ui.pincode, ui.state, ui.zone, ui.actual_city,
    COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,
    bs.scheme_name,
    bs.earned_units,
    mp.points AS earned_point
FROM scan_data sd
LEFT JOIN user_info ui ON ui.id = sd.customer_id
LEFT JOIN box_scans bs ON bs.skui_id = sd.id
LEFT JOIN mypoints mp ON mp.sku_inventory_id = sd.id
    AND mp.users_id = sd.customer_id
WHERE sd.sap_code = 'D12RTM0901';

```

New Categories Specific SE WISE Report

```
SELECT
  sk.status_retailer_id AS retailer_id,
  c.id AS distributor_id,
  m.name AS category,
  count(sk.lpn_number) as scan_count,
  SUM(
    CASE
      WHEN sqpm.box_calculation_uom IS NOT NULL
        AND sqpm.box_calculation_uom > 0
      THEN sqpm.box_calculation_uom
      WHEN sk.uom = 'B5' THEN 0.5
      ELSE 1
    END
  ) AS current_category_box_count,
  sqpm.uom,
  u.name,
  u.mobile_number,
  md.shop_name
FROM sku_inventories sk
```

```

JOIN users u
    ON u.id = sk.status_retailer_id
JOIN sku_products sp
    ON sp.id = sk.product_id
JOIN master_categories m
    ON m.id = sp.categories
JOIN executive_distributor_mappings edm
    ON edm.distributor_id = u.primary_distributor_id
JOIN sku_qr_points_maps sqpm
    ON sqpm.sku_code = sk.sku_code
    AND sqpm.uom = sk.uom
JOIN mechanic_details md
    ON md.mechanic_id = u.id
join companies c on
c.id = u.primary_distributor_id
WHERE u.user_role = 2
    AND edm.executive_id = 53478
    AND sk.retailer_scanned_at >= '2026-04-01'
    AND sk.retailer_scanned_at < '2026-05-01'
    AND NOT EXISTS (
        SELECT 1
        FROM sku_inventories sk2
        JOIN sku_products sp2
            ON sp2.id = sk2.product_id
        WHERE sk2.status_retailer_id = sk.status_retailer_id
            AND sp2.categories = sp.categories
            AND sk2.retailer_scanned_at < '2026-03-01'
    )
GROUP BY
    sk.status_retailer_id,
    sk.distributer_id,
    m.name,
    sk.uom,
    u.name,
    u.mobile_number,
    md.shop_name
ORDER BY
    sk.status_retailer_id,
    m.name;

```

Bonus Active and Inactive data

```
SELECT
  t.user_id,
  t.user_status,
  COUNT(*) AS scanned_boxes,
  ROUND(SUM(t.points) * 0.5) AS bonus_points
FROM
  (
    SELECT
      u.id AS user_id,
      mp.points,
      CASE
        WHEN EXISTS (
          SELECT 1
          FROM sku_inventories si2
          WHERE si2.status_retailer_id = u.id
          AND si2.retailer_scanned_at BETWEEN
            '2025-11-01 00:00:00' AND '2026-01-31 23:59:59'
        )
        THEN 'Active'
        ELSE 'Inactive'
      END AS user_status,
      ROW_NUMBER() OVER (
        PARTITION BY u.id
        ORDER BY si.retailer_scanned_at ASC
      ) AS rn
    FROM users u
    JOIN sku_inventories si
      ON si.status_retailer_id = u.id
      AND si.retailer_scanned_at BETWEEN
        '2026-02-01 00:00:00' AND '2026-02-28 23:59:59'
    JOIN mypoints mp
      ON mp.sku_inventory_id = si.id
      AND mp.users_id = si.status_retailer_id
    WHERE u.user_role = 2
    AND u.created_at < '2025-11-01'
  ) t
WHERE t.rn <= 10
GROUP BY t.user_id, t.user_status
HAVING COUNT(*) >= 10;
```

#adepoli Earning

```
SELECT
  user_id,
  scheme_id,
  SUM(amount) AS balance,

  SUM(CASE
    WHEN reference_type = 'sku_scan'
    THEN amount
    ELSE 0
  END) AS earning,

  abs(SUM(CASE
    WHEN reference_type = 'scheme_gift_order'
    THEN amount
    ELSE 0
  END)) AS redemption
FROM scheme_wallet_transactions w
join schemes s on
w.scheme_id = s.id
WHERE w.scheme_id = 1
GROUP BY user_id, scheme_id
order by balance;
```

#winter Earning

```
SELECT
  DISTINCT user_id,
  SUM(amount) AS balance,
  SUM(CASE
    WHEN reference_type IN ('sku_scan','topup')
    THEN amount
    ELSE 0
  END) AS Earning,
  abs(SUM(CASE
    WHEN reference_type IN ('gift_order','cash_withdrawal','credit_note')
    THEN amount
```

```

        ELSE 0
    END)) AS Redemption
FROM winter_scan_wallet_transactions
GROUP BY
    user_id
ORDER BY
    balance;

```

#namma & majja Earning

```

SELECT
    user_id,
    SUM(amount) AS balance,
    SUM(CASE
        WHEN reference_type IN ('sku_scan','topup')
        THEN amount
        ELSE 0
    END) AS Earning,
    Abs(SUM(CASE
        WHEN reference_type IN ('gift_order','credit_note')
        THEN amount
        ELSE 0
    END)) AS Redemption
FROM gift_scan_wallet_transactions
GROUP BY
    user_id
ORDER BY
    balance;

```

#freedom Earning

```

SELECT
    u.id,
    COALESCE(pb.prev_balance, 0) AS prev_balance,
    COALESCE(w.earn_freedom_balance, 0) AS earning,
    COALESCE(r.redeemed, 0) AS redemption,
    sum(wt.amount) as current_amount,
    ABS(
        CASE
            WHEN COALESCE(pb.prev_balance, 0) - COALESCE(wd.withdrawn, 0) >= 0
            THEN 0
            ELSE COALESCE(pb.prev_balance, 0) - COALESCE(wd.withdrawn, 0)
        END
    ) AS freedom_cash_withdrawal,

```

```

(
    COALESCE(w.earn_freedom_balance, 0)
    - COALESCE(r.redeemed, 0)
    +
    CASE
        WHEN COALESCE(pb.prev_balance, 0)
            - COALESCE(wd.withdrawn, 0) >= 0
        THEN 0
        ELSE COALESCE(pb.prev_balance, 0)
            - COALESCE(wd.withdrawn, 0)
    END
) AS freedom_balance
FROM users u
LEFT JOIN
(
    SELECT user_id,
           SUM(amount) AS earn_freedom_balance
    FROM wallet_transaction
    WHERE reference_type IN ('cash_point', 'topup')
           AND created_at BETWEEN '2025-09-01 00:00:00'
                               AND '2025-11-20 23:59:59'

    GROUP BY user_id
) w ON w.user_id = u.id
left join

wallet_transaction wt on
wt.user_id = u.id

LEFT JOIN
(
    SELECT go.user_id,
           SUM(go.used_points) AS redeemed
    FROM gift_orders go
    INNER JOIN gift_cards gc3
        ON gc3.id = go.gift_card_id
    WHERE go.order_date BETWEEN '2025-09-01 00:00:00'
                               AND '2025-11-20 23:59:59'
           AND gc3.points_applicability = 'date_range'
           AND go.status != 'Cancelled'

    GROUP BY go.user_id
) r ON r.user_id = u.id
LEFT JOIN
(
    SELECT user_id,
           SUM(amount) AS prev_balance
    FROM wallet_transaction
    WHERE created_at < '2025-09-01 00:00:00'

    GROUP BY user_id
) pb ON pb.user_id = u.id

```

```

LEFT JOIN
(
    SELECT user_id,
           SUM(amount) AS withdrawn
    FROM withdrawal_request
    WHERE created_at BETWEEN '2025-09-01 00:00:00'
           AND '2025-11-20 23:59:59'
           AND withdraw_request_type IS NULL
           AND status IN (0,1)
    GROUP BY user_id
) wd ON wd.user_id = u.id
WHERE u.user_role = 2
      AND u.state_id IN (8, 9, 20, 30, 33, 34, 40)
GROUP BY u.id;

```

#Distributor_wallet_data

```

SELECT
    c.id AS distributor_id,
    c.sap_code,
    COALESCE(w.wallet_amount,0) AS wallet_amount,
    COALESCE(d.deactivated_wallet_amount,0) AS deactivated_wallet_amount
FROM users u
JOIN companies c
    ON u.distributor_id = c.id
LEFT JOIN (
    SELECT user_id, SUM(amount) AS wallet_amount
    FROM wallet_transaction
    GROUP BY user_id
) w
    ON w.user_id = u.id
LEFT JOIN (
    SELECT user_id, SUM(amount) AS deactivated_wallet_amount
    FROM deactivated_qr_wallet_transactions
    GROUP BY user_id
) d
    ON d.user_id = u.id
WHERE u.user_role = 4;

```

#check the QR scan

```
SELECT *  
FROM sku_qr_points_maps  
WHERE sku_code IN  
('ASI025BRN#100','ASI025BRN#85','ASI026BRN#95','ASI025BRN#95','ASI026BRN#95');
```

#Check the bank Transactions data for users Transactions

```
SELECT *  
FROM withdrawal_request wt  
JOIN automatic_transactions ats  
    ON wt.user_id = ats.user_id  
JOIN bank_transactions bt  
    ON bt.bankReferenceNumber = ats.bank_reference_number  
WHERE wt.user_id = 73502;
```

```
SELECT *  
FROM withdrawal_request  
where user_id = 73502;
```

```
select * from automatic_transactions where transaction_id = '9FBA22CC952B4E2FBC92C23EB9F2D05B';
```

```
select * from bank_transactions where recieverAccountNumber = 03481000103345;
```

```
select * from bank_accounts where user_id = 73502;
```

#Box scan count of specific retailer

```
select count(uom) from sku_inventories where uom in ('B10','B5')
and status_retailer_id = 48683 and Month(retailer_scanned_at) = 2;
```

Check the wholesaler and distributor report winter and core till march 2026

winter - wholesaler

```
SELECT
    si.status_wholeseller_id,
    SUM(wswt.amount) AS total_amount
FROM sku_inventories si
JOIN sku_products sp
    ON si.product_id = sp.id
JOIN winter_scan_wallet_transactions wswt
    ON si.status_wholeseller_id = wswt.id
WHERE sp.season IN ('winter')
and Date(wholesaler_scanned_at) < '2026-04-01'
GROUP BY si.status_wholeseller_id;
```

#core - wholesaler

```
SELECT
    si.status_wholeseller_id,
    SUM(wswt.amount) AS total_amount
FROM sku_inventories si
JOIN sku_products sp
    ON si.product_id = sp.id
JOIN winter_scan_wallet_transactions wswt
    ON si.status_wholeseller_id = wswt.id
WHERE sp.season IN ('core')
and Date(wholesaler_scanned_at) < '2026-04-01'
GROUP BY si.status_wholeseller_id;
```

```
#coreDistributor__dwt
```

```
#coreDistributor__dwt
```

```
select u.distributer_id,  
sum(wt.amount) as Total_amount  
from users u  
join wallet_transaction wt on  
wt.user_id = u.id  
where u.user_role = 4  
and Date(wt.created_at) < '2026-04-01'  
group by u.distributer_id;
```

Check bank transaction for retailers

```
select sum(amount) from wallet_transaction where user_id = 74442 and reference_type IN(  
'topup','cash_point');
```

```
select sum(amount) from wallet_transaction where user_id = 74442;  
select * from wallet_transaction where user_id = 74442 and reference_type IN( 'topup');
```

```
select * from withdrawal_request where user_id = 74442;
```

```
select * from automatic_transactions where transaction_id = 'BBDEC3C0B1AB4C408B7359A958A82935';
```

```
select * from bank_transactions where decentroUrn = 'BBDEC3C0B1AB4C408B7359A958A82935';
```

Bonus_transactions_data - apr_nov and dec_mar

#apr_nov

```
SELECT
u.id AS customer_id,
md.shop_name,
u.district,
u.pincode,

SUM(CASE WHEN MONTH(bt.created_at)=4 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS APR,
SUM(CASE WHEN MONTH(bt.created_at)=5 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS MAY,
SUM(CASE WHEN MONTH(bt.created_at)=6 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS JUN,
SUM(CASE WHEN MONTH(bt.created_at)=7 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS JUL,
SUM(CASE WHEN MONTH(bt.created_at)=8 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS AUG,
SUM(CASE WHEN MONTH(bt.created_at)=9 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0 END)
AS SEP,
SUM(CASE WHEN MONTH(bt.created_at)=10 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0
END) AS OCT,
SUM(CASE WHEN MONTH(bt.created_at)=11 AND YEAR(bt.created_at)=2025 THEN bt.points ELSE 0
END) AS NOV

FROM users u

LEFT JOIN bonus_transactions bt
    ON bt.user_id = u.id

LEFT JOIN bonus_events be
    ON be.id = bt.bonus_event_id

LEFT JOIN mechanic_details md
    ON u.id = md.mechanic_id

WHERE be.id IN (13,14,15,19,20,21,22) and u.user_role = 2

GROUP BY
u.id,
```

```
md.shop_name,  
u.district,  
u.pincode;
```

dec_mar

```
SELECT  
u.id AS customer_id,  
md.shop_name,  
u.district,  
u.pincode,
```

```
SUM(CASE WHEN MONTH(wt.created_at)=12 AND YEAR(wt.created_at)=2025 THEN wt.amount ELSE 0  
END) AS `DEC`,  
SUM(CASE WHEN MONTH(wt.created_at)=1 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0  
END) AS JAN,  
SUM(CASE WHEN MONTH(wt.created_at)=2 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0  
END) AS FEB,  
SUM(CASE WHEN MONTH(wt.created_at)=3 AND YEAR(wt.created_at)=2026 THEN wt.amount ELSE 0  
END) AS MAR
```

```
FROM wallet_transaction wt
```

```
LEFT JOIN bonus_events be  
ON be.id = wt.reference_id
```

```
LEFT JOIN users u  
on u.id = wt.user_id
```

```
LEFT JOIN mechanic_details md  
ON wt.user_id = md.mechanic_id
```

```
WHERE be.id IN (13,14,15,19,20,21,22) and u.user_role = 2 and wt.reference_type = 'topup'
```

```
GROUP BY  
u.id,  
md.shop_name,  
u.district,
```

u.pincode;

Checking bonus_transactions for specific date

```
select sum(points) from bonus_transactions
join bonus_events on
bonus_transactions.bonus_event_id = bonus_events.id
where bonus_events.id IN (13,14,15,19,20,21,22)
and MONTH(bonus_transactions.created_at) in (4,5,6,7,8,9,10,11)
and bonus_transactions.user_id = 50565;
```

```
select sum(amount) from wallet_transaction
join bonus_events on
wallet_transaction.reference_id = bonus_events.id
where bonus_events.id IN (13,14,15,19,20,21,22)
and wallet_transaction.reference_type = 'topup'
and wallet_transaction.user_id = 50565;
```

Unscanned_data for Winter season

```
SELECT
    u.distributer_id,
    sk.lpn_number,
    sp.season,
    d.amount AS amount,
    sqpm.sku_code,
    sk.sku_description,
    sqpm.uom,
    sp.product_name,
    DATE(d.created_at) as Transaction_date
FROM deactivated_qr_wallet_transactions d
JOIN users u
    ON d.user_id = u.id
JOIN sku_inventories sk
```

```

        ON d.reference_id = sk.id
JOIN sku_products sp
        ON sk.product_id = sp.id AND sp.season = 'winter'
Join sku_qr_points_maps sqpm
on sk.sku_code = sqpm.sku_code

WHERE d.reference_type = 'sku_scan';

```

new retailer_category for the period jan-mar

```

SELECT rcsfeb.* FROM (
SELECT
    skui.status_retailer_id AS retailer_id,
    sp.categories as category_id,
    cat.name as category,

    SUM(
        CASE
            WHEN sqpm.box_calculation_uom IS NOT NULL
            AND sqpm.box_calculation_uom > 0
            THEN sqpm.box_calculation_uom

            WHEN sqpm.box_calculation_uom IS NULL
            OR sqpm.box_calculation_uom = 0
            THEN CASE
                WHEN skui.uom = 'B5' THEN 0.5
                ELSE 1
            END
        )
    ) AS total_box_count

```

```

FROM sku_inventories skui
join sku_products sp on sp.id = skui.product_id
left join master_categories cat on cat.id = sp.categories
join users on users.id = skui.status_retailer_id
left JOIN sku_qr_points_maps sqpm
      ON sqpm.sku_code = skui.sku_code
      AND sqpm.uom = skui.uom

where retailer_scanned_at between '2026-01-01 00:00:00' and '2026-03-12 23:59:59'
and users.created_at < '2026-01-01'

```

```

GROUP BY
      skui.status_retailer_id,
      sp.categories,
      cat.name
) as rcsfeb

```

```

left join (
SELECT
      skui.status_retailer_id AS retailer_id,
      sp.categories as category_id,
      cat.name as category,

      SUM(
      CASE
      WHEN sqpm.box_calculation_uom IS NOT NULL
      AND sqpm.box_calculation_uom > 0
      THEN sqpm.box_calculation_uom

      WHEN sqpm.box_calculation_uom IS NULL
      OR sqpm.box_calculation_uom = 0
      THEN CASE
            WHEN skui.uom = 'B5' THEN 0.5
            ELSE 1
            END
      END
      ) AS total_box_count

```

```

FROM sku_inventories skui

```

```

join sku_products sp on sp.id = skui.product_id
left join master_categories cat on cat.id = sp.categories
join users on users.id = skui.status_retailer_id
left JOIN sku_qr_points_maps sqpm
    ON sqpm.sku_code = skui.sku_code
    AND sqpm.uom = skui.uom

where retailer_scanned_at between '2025-04-01 00:00:00' and '2025-12-31 23:59:59'
and users.created_at < '2026-01-01'

GROUP BY
    skui.status_retailer_id,
    sp.categories,
    cat.name
) as rjan on rjan.retailer_id = rcsfeb.retailer_id and
rjan.category_id = rcsfeb.category_id

where (rjan.total_box_count < 5 OR rjan.total_box_count IS NULL) and rc+sfeb.total_box_count >=5;

```

#Distributor_deactivated wallet amount data

```

SELECT
    c.id AS distributor_id,
    c.sap_code,
    COALESCE(w.wallet_amount,0) AS wallet_amount,
    COALESCE(d.deactivated_wallet_amount,0) AS deactivated_wallet_amount

FROM users u

JOIN companies c
    ON u.distributer_id = c.id

LEFT JOIN (
    SELECT user_id, SUM(amount) AS wallet_amount

```

```

        FROM wallet_transaction
        GROUP BY user_id
    ) w
    ON w.user_id = u.id

LEFT JOIN (
    SELECT user_id, SUM(amount) AS deactivated_wallet_amount
    FROM deactivated_qr_wallet_transactions
    GROUP BY user_id
) d
    ON d.user_id = u.id

WHERE u.user_role = 4;

```

#bonus_Inactive_data for scanned and unscanned

```

SELECT
    t.user_id,
    t.user_status,
    COUNT(*) AS scanned_boxes,
    ROUND(SUM(t.points) * 0.5) AS bonus_points
FROM
    (
        SELECT
            u.id AS user_id,
            mp.points,

            CASE
                WHEN EXISTS (
                    SELECT 1
                    FROM sku_inventories si2
                    WHERE si2.status_retailer_id = u.id
                    AND si2.retailer_scanned_at BETWEEN
                        '2025-11-01 00:00:00' AND '2026-01-31 23:59:59'
                )
                THEN 'Active'
                ELSE 'Inactive'
            END AS user_status,

            ROW_NUMBER() OVER (
                PARTITION BY u.id
                ORDER BY si.retailer_scanned_at ASC
            ) AS rn
    )

```

```

FROM users u

JOIN sku_inventories si
ON si.status_retailer_id = u.id
AND si.retailer_scanned_at BETWEEN
'2026-02-01 00:00:00' AND '2026-02-28 23:59:59'

JOIN mypoints mp
ON mp.sku_inventory_id = si.id
AND mp.users_id = si.status_retailer_id

WHERE u.user_role = 2
AND u.created_at < '2025-11-01'

) t

WHERE t.rn <= 10
GROUP BY t.user_id, t.user_status
HAVING COUNT(*) >= 10;

```

#Welcome_bonus_data for Retailers

```

SELECT
    si.status_retailer_id,
    DATE(u.created_at) AS registered_at,
    COUNT(si.lpn_number) AS feb_box_count,
    MIN(DATE(si.retailer_scanned_at)) AS first_scan_date
FROM users u
JOIN sku_inventories si
    ON u.id = si.status_retailer_id

WHERE u.created_at <= '2025-10-31'

AND si.retailer_scanned_at >= '2026-02-01'
AND si.retailer_scanned_at < '2026-02-01'

AND u.id NOT IN (
    SELECT status_retailer_id
    FROM sku_inventories

```

```
WHERE retailer_scanned_at >= '2025-11-01'  
AND retailer_scanned_at < '2026-02-01'  
)
```

```
GROUP BY u.id, DATE(u.created_at)  
HAVING COUNT(si.lpn_number) > 10;
```

#monthly_points_data and Monthly Avg Points

```
SELECT  
    ma.user_id,  
    SUM(sq.box_calculation_uom) AS total_boxes,  
    ma.monthly_average  
FROM sku_qr_points_maps sq  
JOIN sku_inventories si  
    ON sq.sku_code = si.sku_code  
JOIN mypoints mp  
    ON si.id = mp.sku_inventory_id  
JOIN mapping_incentive_users mpu  
    ON mp.users_id = mpu.user_id  
JOIN monthly_averages ma  
    ON ma.user_id = mpu.user_id  
JOIN incentive_schemes ins  
    ON ma.monthly_average BETWEEN ins.min_val AND ins.max_val  
WHERE ins.status = 1  
AND ins.start_date <= '2026-02-28'  
AND ins.end_date >= '2026-02-01'  
GROUP BY  
    ma.user_id,  
    ma.monthly_average  
ORDER BY mpu.user_id;
```

-- winter wallet scan

```
SELECT
    w.user_id,
    sp.product_name,
    si.sku_code,
    si.sku_description as material_description,
    si.uom,
    si.distributer_id,
    sp.product_code,
    COUNT(si.lpn_number) AS total_lpn,
    SUM(w.amount) AS total_amount,
    DATE(w.created_at) AS scanned_date
FROM sku_inventories si
JOIN winter_scan_wallet_transactions w
    ON si.id = w.reference_id
    AND w.reference_type = 'sku_scan'
JOIN sku_products sp
    ON si.product_id = sp.id
WHERE sp.season = 'winter'
GROUP BY
    w.user_id,
    sp.product_name,
    si.sku_code,
    si.sku_description,
    si.uom,
    si.distributer_id,
    sp.product_code,
    DATE(w.created_at);
```

-- chat support crosscheck category name & id for specific retailer with distributer

```
SELECT
    c.id AS category_id,
    c.name as category_name
FROM sku_inventories si
LEFT JOIN sku_products sp
    ON si.product_id = sp.id
LEFT JOIN master_categories c
```

```

ON c.id = sp.categories
WHERE si.status_retailer_id = 67417
AND si.distributer_id = 6318
AND date(si.retailer_scanned_at) >= '2025-07-01'
AND date(si.retailer_scanned_at) <= NOW()
GROUP BY c.id, c.name;

```

-- bonus point report

```

SELECT b.user_id, b.points, b.created_at, u.user_role
from bonus_transactions b join users u on b.user_id = u.id
where bonus_event_id = 18;

```

-- milestone Report

```

SELECT user_id, reference_id, reference_type, created_at
from wallet_transaction
where reference_type = 'topup' and
created_at >= '2026-03-01' and
reference_id is not null;

```

-- Available Transaction / Balance

```

SELECT
    user_id,
    SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END) -
    SUM(CASE WHEN type = 'debit' THEN ABS(points) ELSE 0 END) AS balance
FROM bonus_transactions
WHERE created_at < '2026-03-01'
GROUP BY user_id;

```

-- first Scan Date

```

select status_retailer_id, min(date(retailer_scanned_at)) from sku_inventories where status_retailer_id
is not null group by status_retailer_id;
SELECT status_retailer_id, MIN(DATE(retailer_scanned_at))
FROM sku_inventories
WHERE status_retailer_id = '46678'
GROUP BY status_retailer_id;

```

```

select id, date(created_at) from users WHERE user_role = 2;

```

-- all combined (Retailer Master DB)

```

SELECT

```

```

    u.id AS user_id,
    DATE(u.created_at) AS register_date,
    wt.balance AS wallet_balance,
    bt.balance AS bonus_balance,
    si.first_scan_date

FROM users u

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS balance
    FROM wallet_transaction
    WHERE created_at < '2026-03-01'
    GROUP BY user_id
) wt ON wt.user_id = u.id

LEFT JOIN (
    SELECT
        user_id,
        SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END) -
        SUM(CASE WHEN type = 'debit' THEN ABS(points) ELSE 0 END) AS balance
    FROM bonus_transactions
    WHERE created_at < '2026-03-01'
    GROUP BY user_id
) bt ON bt.user_id = u.id

LEFT JOIN (
    SELECT
        status_retailer_id,
        MIN(DATE(retailer_scanned_at)) AS first_scan_date
    FROM sku_inventories
    WHERE status_retailer_id IS NOT NULL
    GROUP BY status_retailer_id
) si ON si.status_retailer_id = u.id

WHERE u.user_role = 2
and u.isBlacklisted = 0;

select sum(amount) from wallet_transaction where user_id = 46599
and created_at < '2026-03-01';

```

-- Retailer basic list + state name from state table

```
select u.id, u.name, u.mobile_number, st.sname from users u left join state st on u.state_id = st.id
where u.user_role = 2;
```

-- Retailer Shop Name From MeCHANIC tABLE + uSERS

```
select u.id, m.shop_name from users u left join mechanic_details m on u.id = m.id
where u.user_role = 2;
```

-- Total Box Scanned for Distributer Report (Jan - Mar)

```
SELECT
    distributor_id,
    status_retailer_id,

    -- Monthly totals
    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 1 THEN
            CASE
                WHEN uom = 'B10' THEN 1
                WHEN uom = 'B5' THEN 0.5
            ELSE 0
            END
        ELSE 0
    END) AS jan_boxes,

    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 2 THEN
            CASE
                WHEN uom = 'B10' THEN 1
                WHEN uom = 'B5' THEN 0.5
            ELSE 0
            END
        ELSE 0
    END) AS feb_boxes,

    SUM(CASE
        WHEN MONTH(retailer_scanned_at) = 3 THEN
```

```

CASE
WHEN uom = 'B10' THEN 1
WHEN uom = 'B5' THEN 0.5
ELSE 0
END
ELSE 0
END) AS mar_boxes,

-- Total for 3 months
SUM(
CASE
WHEN uom = 'B10' THEN 1
WHEN uom = 'B5' THEN 0.5
ELSE 0
END
) AS total_boxes_Jan_Mar

FROM sku_inventories

WHERE retailer_scanned_at >= '2026-01-01 00:00:00'
AND retailer_scanned_at < '2026-04-01 00:00:00'

GROUP BY distributer_id, status_retailer_id

ORDER BY distributer_id, status_retailer_id;

-- FINAL Winter distributor unscanned report
SELECT
    u.distributer_id,
    sk.lpn_number,
    sp.season,
    d.amount AS amount
FROM deactivated_qr_wallet_transactions d
JOIN users u
    ON d.user_id = u.id
JOIN sku_inventories sk
    ON d.reference_id = sk.id
JOIN sku_products sp
    ON sk.product_id = sp.id AND sp.season = 'winter'
WHERE d.reference_type = 'sku_scan'

```

-- to check cashpoint & bonus (top up)

```
select * from users where mobile_number = 9911570413;
```

```
SELECT *  
FROM wallet_transaction  
WHERE user_id = 53756  
AND reference_type IN ('cash_point', 'topup')  
AND created_at >= '2025-09-01'  
AND created_at < '2026-04-15';
```

– Total Earnings and Credit Note for Distributers

```
SELECT  
    wt.user_id, sum(wt.amount) as Total_amount,
```

-- Total Earnings

```
    SUM(  
        CASE  
            WHEN wt.reference_type IN ('cash_point','topup','referral_earning','incentive')  
            THEN wt.amount  
            ELSE 0  
        END  
    ) AS Total_Earnings,
```

-- Credit Note

```
    SUM(  
        CASE  
            WHEN wt.reference_type = 'credit_note'  
            THEN wt.amount  
            ELSE 0  
        END  
    ) AS Credit_Note
```

```
FROM wallet_transaction wt  
JOIN users u  
    ON wt.user_id = u.id
```

```
WHERE  
    wt.created_at < '2026-04-01' -- Till 31 st March  
    AND u.user_role = 4
```

```
GROUP BY wt.user_id;
```

-- UOM BOX SCANNED

```
select COUNT(uom) from sku_inventories
where status_retailer_id = 75086
and uom = 'B5'
and retailer_scanned_at >= '2026-04-01'
and retailer_scanned_at < '2026-04-17';
```

```
select * from sku_inventories
where status_retailer_id = 75086
and uom in ('B5','B10');
```

-- Deactivated Unscanned Report for Distributer

```
SELECT
    u.distributer_id,

    SUM(
        CASE
            WHEN dqwt.reference_type IN ('sku_scan')
            AND sp.season = 'winter'
            THEN dqwt.amount
            ELSE 0
        END
    ) AS Winter_Earning,
    SUM(
        CASE
            WHEN dqwt.reference_type IN ('sku_scan')
            AND ( sp.season = 'core' or sp.season is null)
            THEN dqwt.amount
            ELSE 0
        END
    ) AS core_Earning,

    SUM(dqwt.amount) AS Balance,

    SUM(
        CASE
            WHEN dqwt.reference_type IN ('topup')
            THEN dqwt.amount
```

```

ELSE 0
END
) AS Topup_Earning,
ABS (SUM(
CASE
WHEN dqwt.reference_type IN ('credit_note') -- abs
THEN dqwt.amount
ELSE 0
END
) )AS credit_note

FROM deactivated_qr_wallet_transactions dqwt

JOIN users u
    ON u.id = dqwt.user_id

LEFT JOIN sku_inventories si
    ON si.id = dqwt.reference_id

LEFT JOIN sku_products sp
    ON sp.id = si.product_id

WHERE dqwt.created_at < '2026-04-01'
AND u.user_role = 4
GROUP BY u.distributer_id;

```

-- Credit Note for retailer & wholesaler report

```

SELECT u.`name` as name,c.`name`as gift_name,
so.shop_name,so.order_date,so.used_points,so.status,so.order_id,so.user_id,so.cn_amount,so.cn_date
FROM scheme_gift_orders so
join gift_cards c on so. = c.id
join users u on so.user_id = u.id
WHERE so.status = 'CN adjusted'
and u.user_role in (2,5);

```

-- Distributer Unsacned Wallet Point Yearly Report

```

SELECT
    u.distributer_id,
    DATE(dqwt.created_at) AS created_date,
    si.lpn_number,

```

```
si.sku_description,  
si.sku_code,  
sq.uom,  
sq.gride_type,  
sq.box_calculation_uom as Box_calculation,
```

```
SUM(  
CASE  
WHEN dqwt.reference_type = 'sku_scan'  
AND sp.season = 'winter'  
THEN dqwt.amount  
ELSE 0  
END  
) AS Winter_Earning,
```

```
SUM(  
CASE  
WHEN dqwt.reference_type = 'sku_scan'  
AND (sp.season = 'core' OR sp.season IS NULL)  
THEN dqwt.amount  
ELSE 0  
END  
) AS Core_Earning,
```

```
SUM(dqwt.amount) AS Balance,
```

```
SUM(  
CASE  
WHEN dqwt.reference_type = 'topup'  
THEN dqwt.amount  
ELSE 0  
END  
) AS Topup_Earning,
```

```
ABS(SUM(  
CASE  
WHEN dqwt.reference_type = 'credit_note'  
THEN dqwt.amount  
ELSE 0  
END  
)) AS Credit_Note
```

```

FROM deactivated_qr_wallet_transactions dqwt
JOIN users u
    ON u.id = dqwt.user_id

LEFT JOIN sku_inventories si
    ON si.id = dqwt.reference_id AND dqwt.reference_type = 'sku_scan'

LEFT JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sq
    ON si.sku_code = sq.sku_code
    AND si.uom = sq.uom

WHERE dqwt.created_at < '2026-04-01'
AND u.user_role = 4

GROUP BY
    u.distributer_id,
    DATE(dqwt.created_at),
    si.lpn_number,
    si.sku_description,
    si.sku_code,
    sq.uom,
    sq.gride_type,
    sq.box_calculation_uom;

```

– Distributor Targets Report (Quarter Wise)

Select * FROM companies ;

– To check Redeemtion

```

select user_id , sum(amount) from scheme_wallet_transactions
where scheme_id = 3
and reference_type in ('scheme_gift_order','cash_convert')
group by user_id ;

```

– To check Balance and Earnings

SELECT

```

user_id,
SUM(CASE
    WHEN reference_type = 'sku_scan'
    THEN amount
    ELSE 0
END) AS total_earning,
SUM(amount) AS total_balance
FROM scheme_wallet_transactions
GROUP BY user_id;

```

-- bonus Balance and Cash Balance

```

SUM(CASE WHEN type = 'credit' THEN points ELSE 0 END)
-
SUM(CASE WHEN type = 'Debit' THEN points ELSE 0 END)
AS bonus_balance

```

-- for retailers

```

SELECT
    u.id,
    u.name,
    u.mobile_number,
    COALESCE(w.cash_balance, 0) AS Cash_Balance,

    COALESCE(b.bonus_balance, 0) AS Bonus_Balance

FROM users u

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS cash_balance
    FROM wallet_transaction
    WHERE DATE(created_at) <= '2026-04-30'
    GROUP BY user_id
) w ON u.id = w.user_id

LEFT JOIN (
    SELECT

```

```

        user_id,
        SUM(CASE
            WHEN type = 'credit' THEN points
            WHEN type = 'Debit' THEN -points
            ELSE 0
        END) AS bonus_balance
    FROM bonus_transactions
    GROUP BY user_id
) b ON u.id = b.user_id

```

```
WHERE u.user_role = 2;
```

-- For Wholesaler only cash balance

```

SELECT
    u.id,
    u.name,
    u.mobile_number,
    COALESCE(w.cash_balance, 0) AS Cash_Balance,

```

```
FROM users u
```

```

LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS cash_balance
    FROM wallet_transaction
    WHERE DATE(created_at) <= '2026-04-30'
    GROUP BY user_id
) w ON u.id = w.user_id

```

```
WHERE u.user_role = 5;
```

-- retaier scan mapped / not - checking!!

```

SELECT * FROM sku_inventories WHERE lpn_number IN
('UASVMAQ','FULNVGE','OGAZYIG','HHBKBGL','CNKVRIL'); -- here take sku_code
SELECT * FROM sku_qr_points_maps where sku_code = "ACM001BRN2PC#90";

```

-- Growth Scheme (unit_scanned , Points, bonus , growth, total earned points)

```

SELECT
    u.id,
    u.state_id,
    u.city,
    u.user_role,
    u.pincode,
    m.monthly_average,

    COALESCE(us.Unit_Scanned, 0) AS Unit_Scanned,
    COALESCE(mp.Points, 0) AS Points,
    COALESCE(bp.Bonus_Points, 0) AS Bonus_Points,
    COALESCE(gp.Growth_Points, 0) AS Growth_Points,
    (
        COALESCE(bp.Bonus_Points, 0) +
        COALESCE(gp.Growth_Points, 0)
    ) AS Total_Points_Earned

```

```

FROM users u

```

```

JOIN monthly_averages m
    ON u.id = m.user_id

```

```

-- Unit Scanned

```

```

LEFT JOIN (
    SELECT
        si.status_retailer_id ,
        SUM(sq.box_calculation_uom) AS Unit_Scanned
    FROM sku_inventories si
    JOIN sku_qr_points_maps sq
    ON si.uom = sq.uom
    AND si.sku_code = sq.sku_code
    GROUP BY si.status_retailer_id
) us
    ON u.id = us.status_retailer_id

```

```

-- Total Points

```

```

LEFT JOIN (
    SELECT
        users_id,
        SUM(points) AS Points
    FROM mypoints
    WHERE created_at >= '2026-03-01'

```

```

        AND created_at < '2026-04-01'
        GROUP BY users_id
    ) mp
    ON u.id = mp.users_id

-- Bonus Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Bonus_Points
    FROM wallet_transaction
    WHERE reference_type = 'topup'
    AND reference_id REGEXP '^(13|19|20|21|22)'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) bp
    ON u.id = bp.user_id

-- Growth Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Growth_Points
    FROM wallet_transaction
    WHERE reference_type = 'incentive'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) gp
    ON u.id = gp.user_id

```

```
WHERE u.user_role = 2;
```

– Toing_scheme_karnataka

```

SELECT
    si.distributer_id,
    u.`name`,
    si.status_retailer_id,
    si.retailer_scanned_at,
    sp.product_name,

```

```

sp.product_code,

-- Earned Units (only sku_scan)
COALESCE(SUM(
    CASE
        WHEN wt.reference_type = 'sku_scan'
        THEN wt.amount
        ELSE 0
    END
), 0) AS Earned_Units,
csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id

JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code
    AND sqpm.uom = si.uom

JOIN category_sub_groups csg
    ON csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id
    AND wt.scheme_id = 3 -- keep scheme filter here

WHERE u.user_role = 2
AND u.id = 50547

GROUP BY
    si.distributer_id,
    u.name,
    si.status_retailer_id,
    si.retailer_scanned_at,
    sp.product_name,
    sp.product_code,
    csg.name;

```

– toing scheme karnataka with more columns

```
SELECT
    si.distributer_id,
    u.`name`,
    u.mobile_number,
    si.status_retailer_id,
    si.retailer_scanned_at,
    sp.product_name,
    sp.product_code,
    si.lpn_number,
    si.uom,
    sqpm.gride_type,
    SUM(sqpm.box_calculation_uom) AS Total_Box_Cal_UOM,

    -- Earned Units (only sku_scan)
    COALESCE(SUM(
        CASE
            WHEN wt.reference_type = 'sku_scan'
            THEN wt.amount
            ELSE 0
        END
    ), 0) AS Earned_Units,
    csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id

JOIN sku_products sp
    ON sp.id = si.product_id

LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code
    AND sqpm.uom = si.uom

JOIN category_sub_groups csg
    ON csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
```

```
ON si.id = wt.reference_id
AND wt.scheme_id = 3 -- keep scheme filter here
```

```
WHERE u.user_role = 2
AND u.id = 74505
```

```
GROUP BY
  si.distributer_id,
  u.name,
  u.mobile_number,
  si.status_retailer_id,
  si.retailer_scanned_at,
  sp.product_name,
  sp.product_code,
  si.uom,
  csg.name,
  si.lpn_number,
  sqpm.gride_type;
```

-- Deactivated unscanned report (open balance , C.N - winter & core)

Balance (calc manually in sheet) = Open_winter_Balance + Open_core_Balance +
Winter_Earning + Core_Earning + Topup_Earning - Credit_Note_winter + Credit_Note_core (C N)

```
SELECT
  t.*,
  -- Core Balance
  (
    t.Open_core_Balance
    + t.Core_Earning
    + t.Topup_Earning
    - t.Credit_Note_core
  ) AS Core_Balance,

  -- Winter Balance
  (
    t.Open_winter_Balance
    + t.Winter_Earning
    + t.Topup_Earning
```

```
- t.Credit_Note_winter  
) AS Winter_Balance
```

```
FROM
```

```
(
```

```
SELECT
```

```
u.distributer_id,
```

```
-- Opening Balance (Before April 1)
```

```
SUM(  
  CASE
```

```
    WHEN dqwt.created_at < '2026-04-01'
```

```
    THEN dqwt.amount
```

```
    ELSE 0
```

```
  END
```

```
) AS Open_Balance,
```

```
-- Opening Winter Balance
```

```
SUM(  
  CASE
```

```
    WHEN dqwt.created_at < '2026-04-01'
```

```
      AND sp.season = 'winter'
```

```
    THEN dqwt.amount
```

```
    ELSE 0
```

```
  END
```

```
) AS Open_winter_Balance,
```

```
-- Opening Core Balance
```

```
SUM(  
  CASE
```

```
    WHEN dqwt.created_at < '2026-04-01'
```

```
      AND (sp.season = 'core' OR sp.season IS NULL)
```

```
    THEN dqwt.amount
```

```
    ELSE 0
```

```
  END
```

```
) AS Open_core_Balance,
```

```
-- Winter Earnings (April - June)
```

```
SUM(  
  CASE
```

```
    WHEN dqwt.created_at >= '2026-04-01'
```

```
      AND dqwt.created_at < '2026-07-01'
```

```

        AND dqwt.reference_type = 'sku_scan'
        AND sp.season = 'winter'
        THEN dqwt.amount
        ELSE 0
    END
) AS Winter_Earning,

-- Core Earnings (April - June)
SUM(
    CASE
        WHEN dqwt.created_at >= '2026-04-01'
        AND dqwt.created_at < '2026-07-01'
        AND dqwt.reference_type = 'sku_scan'
        AND (sp.season = 'core' OR sp.season IS NULL)
        THEN dqwt.amount
        ELSE 0
    END
) AS Core_Earning,

-- Topup Earnings (April - June)
SUM(
    CASE
        WHEN dqwt.created_at >= '2026-04-01'
        AND dqwt.created_at < '2026-07-01'
        AND dqwt.reference_type = 'topup'
        THEN dqwt.amount
        ELSE 0
    END
) AS Topup_Earning,

-- Total Credit Note
ABS(
    SUM(
        CASE
            WHEN dqwt.created_at >= '2026-04-01'
            AND dqwt.created_at < '2026-07-01'
            AND dqwt.reference_type = 'credit_note'
            THEN dqwt.amount
            ELSE 0
        END
    )
) AS Credit_Note,

```

-- Winter Credit Note

```
ABS(  
  SUM(  
    CASE  
      WHEN dqwt.created_at >= '2026-04-01'  
      AND dqwt.created_at < '2026-07-01'  
      AND dqwt.reference_type = 'credit_note'  
      AND dqwt.season = 'winter'  
      THEN dqwt.amount  
      ELSE 0  
    END  
  )  
) AS Credit_Note_winter,
```

-- Core Credit Note

```
ABS(  
  SUM(  
    CASE  
      WHEN dqwt.created_at >= '2026-04-01'  
      AND dqwt.created_at < '2026-07-01'  
      AND dqwt.reference_type = 'credit_note'  
      AND dqwt.season = 'core'  
      THEN dqwt.amount  
      ELSE 0  
    END  
  )  
) AS Credit_Note_core
```

```
FROM deactivated_qr_wallet_transactions dqwt  
JOIN users u  
  ON u.id = dqwt.user_id  
LEFT JOIN sku_inventories si  
  ON si.id = dqwt.reference_id  
LEFT JOIN sku_products sp  
  ON sp.id = si.product_id  
WHERE u.user_role = 4  
GROUP BY u.distributer_id  
) AS t;
```

-- Retailers Scan count(UOM) & Cash_point

```
select user_id , sum(amount), COUNT(created_at) from wallet_transaction
where user_id in (

)
and reference_type IN ('cash_point','topup')
and created_at >= '2026-06-01'
and created_at < '2026-07-01'
group by user_id;
```

-- Retailer scan unit point and scan count (not cash point)

```
select si.status_retailer_id,sum(sq.box_calculation_uom),count(si.retailer_scanned_at)
FROM sku_inventories si
JOIN sku_qr_points_maps sq
    ON si.sku_code = sq.sku_code and si.uom = sq.uom
where si.retailer_scanned_at >= '2026-06-01'
    AND si.retailer_scanned_at < '2026-07-01'
AND si.status_retailer_id in (74417) group by si.status_retailer_id;
```

-- retailers list who has not scanned for last 2 months

```
SELECT
    u.id,
    u.name,
    u.city,
    u.pincode,
    u.address,
    u.email,
    u.mobile_number
FROM users u
WHERE u.state_id = 8
    AND u.user_role = 2
    AND NOT EXISTS (
        SELECT 1
        FROM sku_inventories si
        WHERE si.status_retailer_id = u.id
            AND si.retailer_scanned_at >= '2026-05-01'
            AND si.retailer_scanned_at < '2026-07-01'
    );
```

-- New Retailers Onboarded list for May Month

```
WITH distributor_base AS (  
    SELECT DISTINCT distributor_id  
    FROM executive_distributor_mappings  
    WHERE executive_id = 53478  
)  
  
new_retailers AS (  
    SELECT  
        u.id AS retailer_id,  
        md.shop_name,  
        u.mobile_number,  
        u.mobile_number AS whatsapp_number,  
        u.city,  
        d.district_name,  
        u.created_at AS onboarded_date  
    FROM users u  
    LEFT JOIN mechanic_details md  
        ON md.mechanic_id = u.id  
    LEFT JOIN pincode_address_maps pam  
        ON pam.pincode = u.pincode  
    LEFT JOIN districts d  
        ON d.id = pam.district_id  
    LEFT JOIN sku_inventories si  
        ON si.status_retailer_id = u.id  
    WHERE u.primary_distributor_id IN (  
        SELECT distributor_id  
        FROM distributor_base  
    )  
    AND u.user_role = 2  
    AND u.status = 'approved'  
    AND u.created_at >= '2026-03-01'  
    AND u.created_at < '2026-04-01'  
    AND si.retailer_scanned_at >= '2026-03-01' -- this throws duplicate here  
    AND si.retailer_scanned_at < '2026-04-01'  
)
```

```

scan_data AS (
  SELECT
    si.status_retailer_id AS retailer_id,
    si.sku_code,
    si.uom,
    CASE
      WHEN sqm.box_calculation_uom IS NOT NULL
        AND sqm.box_calculation_uom > 0
        THEN sqm.box_calculation_uom
      WHEN si.uom = 'B5'
        THEN 0.5
      ELSE 1
    END AS box_value
  FROM sku_inventories si
  LEFT JOIN sku_qr_points_maps sqm
    ON sqm.sku_code = si.sku_code
    AND sqm.uom = si.uom
  WHERE si.retailer_scanned_at >= '2026-03-01'
    AND si.retailer_scanned_at < '2026-04-01'
),

```

```

retailer_boxes AS (
  SELECT
    retailer_id,
    COUNT(*) AS scan_count,
    SUM(box_value) AS scanned_boxes
  FROM scan_data
  GROUP BY retailer_id
)

```

```

SELECT
  nr.retailer_id,
  nr.shop_name,
  nr.mobile_number,
  nr.whatsapp_number,
  nr.city,
  nr.district_name,
  DATE(nr.onboarded_date) AS onboarded_date,
  COALESCE(rb.scan_count, 0) AS scan_count,
  COALESCE(rb.scanned_boxes, 0) AS scanned_boxes
FROM new_retailers nr
LEFT JOIN retailer_boxes rb

```

```
ON rb.retailer_id = nr.retailer_id  
ORDER BY nr.onboarded_date;
```

-- bonus where remark = only bonus then use wallet transaction

```
select wt.* FROM wallet_transaction wt join users u on u.id = wt.user_id  
WHERE wt.reference_type in ('topup', 'incentive')  
AND reference_id is not null  
AND wt.created_at >= '2026-07-01 00:00:00'  
AND wt.created_at < '2026-08-05 00:00:00'  
and u.user_role = 2;
```

-- Monthly average for retailers

```
select m.* FROM monthly_averages m join users u on u.id = m.user_id  
WHERE u.user_role = 2;
```

JHG Field Sales (php to SQL queries)

– Onboarding Targets

```
WITH quarter_target_cte AS (  
    SELECT  
        ot.se_id,  
        CASE  
            WHEN 3 BETWEEN 4 AND 6 THEN ot.q1_target  
            WHEN 3 BETWEEN 7 AND 9 THEN ot.q2_target  
            WHEN 3 BETWEEN 10 AND 12 THEN ot.q3_target  
            ELSE ot.q4_target
```

```

        END AS quarter_target
        FROM onboarding_targets ot
        WHERE ot.se_id = 53479
    ),

    monthly_target_cte AS (
        SELECT
            COALESCE(quarter_target / 3, 0) AS monthly_target
        FROM quarter_target_cte
    ),

    distributor_base AS (
        SELECT DISTINCT distributor_id
        FROM executive_distributor_mappings
        WHERE executive_id = 53479
    ),

    new_retailers AS (
        SELECT
            u.id AS retailer_id,
            md.shop_name,
            u.mobile_number,
            u.mobile_number AS whatsapp_number,
            u.city,
            d.district_name
        FROM users u
        LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
        LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode
        LEFT JOIN districts d ON d.id = pam.district_id
        WHERE u.primary_distributor_id IN (SELECT distributor_id FROM distributor_base)
        AND u.user_role = 2
        AND u.status = 'approved'
        AND u.created_at BETWEEN '2026-03-01 00:00:00' AND '2026-03-31 23:59:59'
    ),

    scan_data AS (
        SELECT
            si.status_retailer_id AS retailer_id,
            si.uom,
            CASE
                WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0
                THEN sqm.box_calculation_uom
                WHEN si.uom = 'B5'

```

```

        THEN 0.5
        ELSE 1
        END AS box_value
    FROM sku_inventories si
    LEFT JOIN sku_qr_points_maps sqm
    ON si.sku_code = sqm.sku_code
    AND si.uom = sqm.uom
    WHERE si.retailer_scanned_at BETWEEN '2026-03-01 00:00:00'
        AND '2026-03-31 23:59:59'
),

retailer_boxes AS (
    SELECT
        nr.retailer_id,
        nr.shop_name,
        nr.mobile_number,
        nr.whatsapp_number,
        nr.city,
        nr.district_name,
        COALESCE(SUM(sd.box_value), 0) AS scanned_boxes
    FROM new_retailers nr
    LEFT JOIN scan_data sd
    ON sd.retailer_id = nr.retailer_id
    GROUP BY
        nr.retailer_id,
        nr.shop_name,
        nr.mobile_number,
        nr.whatsapp_number,
        nr.city,
        nr.district_name
),

qualified_retailers AS (
    SELECT *
    FROM retailer_boxes
    WHERE scanned_boxes >= 10
)
SELECT
    (SELECT monthly_target FROM monthly_target_cte LIMIT 1) AS monthly_target,

    -- Monthly achievement
    (SELECT COUNT(*) FROM new_retailers) AS monthly_achievement,

```

```

-- All retailers
(SELECT COUNT(*) FROM retailer_boxes) AS total_new_retailers,
(SELECT COALESCE(SUM(scanned_boxes),0) FROM retailer_boxes) AS total_boxes_all,

-- Qualified retailers
(SELECT COUNT(*) FROM qualified_retailers) AS qualified_retailers_count,
(SELECT COALESCE(SUM(scanned_boxes),0) FROM qualified_retailers) AS qualified_total_boxes;

```

-- new category

```

WITH TargetRetailers AS (
    -- 1. Get the Executive Auth and their associated Approved Retailers
    SELECT u.id AS retailer_id
    FROM users u
    WHERE u.primary_distributor_id IN (
        SELECT distributor_id
        FROM executive_distributor_mappings
        WHERE executive_id = 53479
    )
    AND u.user_role = 2
    AND u.status = 'approved'
),
PrevData AS (
    -- 2. Aggregate Previous Data: From Financial Year Start (April 1st) string calculated dynamically
    --> until Start of Current Month
    SELECT
        si.status_retailer_id as retailer_id,
        sp.categories as category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
                    sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) as prev_total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
    JOIN users u ON u.id = si.status_retailer_id

```

```

WHERE u.created_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00') -- Created before this
month
-- FY Date Logic mimicking PHP $fyStartDate
AND si.retailer_scanned_at >= STR_TO_DATE(CONCAT(CASE WHEN MONTH(NOW()) >= 4 THEN
YEAR(NOW()) ELSE YEAR(NOW()) - 1 END, '-04-01'), '%Y-%m-%d')
-- Up to the end of last month
AND si.retailer_scanned_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
GROUP BY si.status_retailer_id, sp.categories
),
CurrentData AS (
-- 3. Aggregate Current Data (This month till date)
SELECT
si.status_retailer_id as retailer_id,
sp.categories as category_id,
SUM(
CASE
WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
sqm.box_calculation_uom
WHEN si.uom = 'B5' THEN 0.5
ELSE 1
END
) as current_total_boxes
FROM sku_inventories si
JOIN sku_products sp ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
JOIN users u ON u.id = si.status_retailer_id
WHERE u.created_at < DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
-- From start of this month till right NOW
AND si.retailer_scanned_at >= DATE_FORMAT(NOW(), '%Y-%m-01 00:00:00')
AND si.retailer_scanned_at <= NOW()
GROUP BY si.status_retailer_id, sp.categories
),
NewCategoryActivations AS (
-- 4. Apply The Business Logic (Current Data where Previous Data was < 5 or NULL)
SELECT
c.retailer_id,
c.category_id,
c.current_total_boxes,
mc.name as category_name,
CASE WHEN c.current_total_boxes >= 5 THEN 1 ELSE 0 END as is_qualified
FROM CurrentData c

```

```

LEFT JOIN PrevData p ON c.retailer_id = p.retailer_id AND c.category_id = p.category_id
JOIN master_categories mc ON mc.id = c.category_id
WHERE (p.prev_total_boxes IS NULL OR p.prev_total_boxes < 5)
)
-- 5. Final Output joined with Retailer profiles
SELECT
    nca.retailer_id,
    md.shop_name,
    u.mobile_number,
    u.mobile_number as whatsapp_number,
    u.city,
    nca.category_id,
    nca.category_name,
    nca.current_total_boxes as boxes_scanned,
    nca.is_qualified
FROM NewCategoryActivations nca
JOIN users u ON u.id = nca.retailer_id
LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
ORDER BY nca.retailer_id, nca.category_id;

```

-- incremental sales

```

WITH distributor_base AS (
    SELECT distributor_id
    FROM executive_distributor_mappings
    WHERE executive_id = 53479
),

/* -----
DATE CONSTANTS
-----*/
date_bounds AS (
    SELECT
        '2026-02-01 00:00:00' AS start_date,
        '2026-02-28 23:59:59' AS end_date,
        '2025-04-01 00:00:00' AS fy_start,
        '2026-01-31 23:59:59' AS last_month_end
),

/* -----
NEW RETAILERS (MARCH)
-----*/

```

```

new_retailers AS (
    SELECT id
    FROM users
    WHERE primary_distributor_id IN (SELECT distributor_id FROM distributor_base)
    AND user_role = 2
    AND status = 'approved'
    AND created_at BETWEEN (SELECT start_date FROM date_bounds)
        AND (SELECT end_date FROM date_bounds)
),

```

```

/* -----
CURRENT MONTH DATA (NEW RETAILERS)
-----*/

```

```

current_new_data AS (
    SELECT
        si.status_retailer_id AS retailer_id,
        sp.categories AS category_id,
        SUM(
            CASE
                WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom
                WHEN si.uom = 'B5' THEN 0.5
                ELSE 1
            END
        ) AS total_boxes
    FROM sku_inventories si
    JOIN sku_products sp ON sp.id = si.product_id
    LEFT JOIN sku_qr_points_maps sqm
        ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
    WHERE si.status_retailer_id IN (SELECT id FROM new_retailers)
    AND si.retailer_scanned_at BETWEEN
        (SELECT start_date FROM date_bounds)
        AND (SELECT end_date FROM date_bounds)
    GROUP BY si.status_retailer_id, sp.categories
),

```

```

/* -----
QUALIFIED RETAILERS (>=10 BOXES)
-----*/

```

```

qualified_retailers AS (
    SELECT retailer_id, SUM(total_boxes) AS total_boxes
    FROM current_new_data
    GROUP BY retailer_id
)

```

```

        HAVING SUM(total_boxes) >= 10
    ),

    qualified_total AS (
        SELECT SUM(total_boxes) AS total_boxes
        FROM qualified_retailers
    ),

    /* -----
    OLD RETAILERS
    -----*/
    old_retailers AS (
        SELECT id
        FROM users
        WHERE primary_distributor_id IN (SELECT distributor_id FROM distributor_base)
        AND user_role = 2
        AND status = 'approved'
        AND created_at < (SELECT start_date FROM date_bounds)
    ),

    /* -----
    PREVIOUS CATEGORY DATA
    -----*/
    prev_category_data AS (
        SELECT
            si.status_retailer_id AS retailer_id,
            sp.categories AS category_id,
            SUM(
                CASE
                    WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom
                    WHEN si.uom = 'B5' THEN 0.5
                    ELSE 1
                END
            ) AS total_boxes
        FROM sku_inventories si
        JOIN sku_products sp ON sp.id = si.product_id
        LEFT JOIN sku_qr_points_maps sqm
            ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
        WHERE si.status_retailer_id IN (SELECT id FROM old_retailers)
        AND si.retailer_scanned_at BETWEEN
            (SELECT fy_start FROM date_bounds)
            AND (SELECT last_month_end FROM date_bounds)
    )

```

```

        GROUP BY si.status_retailer_id, sp.categories
    ),

    /* -----
    CURRENT CATEGORY DATA
    -----*/
    current_category_data AS (
        SELECT
            si.status_retailer_id AS retailer_id,
            sp.categories AS category_id,
            SUM(
                CASE
                    WHEN sqm.box_calculation_uom > 0 THEN sqm.box_calculation_uom
                    WHEN si.uom = 'B5' THEN 0.5
                    ELSE 1
                END
            ) AS total_boxes
        FROM sku_inventories si
        JOIN sku_products sp ON sp.id = si.product_id
        LEFT JOIN sku_qr_points_maps sqm
            ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
        WHERE si.status_retailer_id IN (SELECT id FROM old_retailers)
        AND si.retailer_scanned_at BETWEEN
            (SELECT start_date FROM date_bounds)
            AND (SELECT end_date FROM date_bounds)
        GROUP BY si.status_retailer_id, sp.categories
    ),

    /* -----
    NEW CATEGORY BOXES
    -----*/
    new_category_boxes AS (
        SELECT SUM(c.total_boxes) AS total_boxes
        FROM current_category_data c
        LEFT JOIN prev_category_data p
            ON c.retailer_id = p.retailer_id
            AND c.category_id = p.category_id
        WHERE
            (p.total_boxes IS NULL AND c.total_boxes >= 5)
            OR (p.total_boxes < 5)
    )

```

```
/* -----  
FINAL OUTPUT  
-----*/
```

```
SELECT  
    (SELECT total_boxes FROM qualified_total) +  
    (SELECT total_boxes FROM new_category_boxes) AS total_incremental_sales,  
  
    (SELECT total_boxes FROM qualified_total)  
    AS total_boxes_by_qualified_retailers,  
  
    (SELECT total_boxes FROM new_category_boxes)  
    AS total_box_scanned_from_new_category;
```

-- new category april1 - april 17

```
WITH TargetRetailers AS (  
    SELECT u.id AS retailer_id  
    FROM users u  
    WHERE u.primary_distributor_id IN (  
        SELECT distributor_id  
        FROM executive_distributor_mappings  
        WHERE executive_id = 53509  
    )  
    AND u.user_role = 2  
    AND u.status = 'approved'  
) ,  
  
PrevData AS (  
    -- From FY start (April 1, 2026) → before April 1, 2026 (no change in logic intent)  
    SELECT  
        si.status_retailer_id as retailer_id,  
        sp.categories as category_id,  
        SUM(  
            CASE  
                WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN  
sqm.box_calculation_uom  
                WHEN si.uom = 'B5' THEN 0.5  
            ELSE 1  
            END  
        ) as prev_total_boxes  
    FROM sku_inventories si
```

```

JOIN sku_products sp ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
JOIN users u ON u.id = si.status_retailer_id
WHERE u.created_at < '2026-04-01 00:00:00'
AND si.retailer_scanned_at >= '2026-04-01 00:00:00'
AND si.retailer_scanned_at < '2026-04-01 00:00:00'
GROUP BY si.status_retailer_id, sp.categories
),

```

```

CurrentData AS (
    -- April 1 → April 17, 2026
    SELECT
        si.status_retailer_id as retailer_id,
        sp.categories as category_id,
SUM(
    CASE
        WHEN sqm.box_calculation_uom IS NOT NULL AND sqm.box_calculation_uom > 0 THEN
sqm.box_calculation_uom
        WHEN si.uom = 'B5' THEN 0.5
        ELSE 1
    END
) as current_total_boxes
FROM sku_inventories si
JOIN sku_products sp ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqm ON si.sku_code = sqm.sku_code AND si.uom = sqm.uom
JOIN TargetRetailers tr ON tr.retailer_id = si.status_retailer_id
JOIN users u ON u.id = si.status_retailer_id
WHERE u.created_at < '2026-04-01 00:00:00'
AND si.retailer_scanned_at >= '2026-04-01 00:00:00'
AND si.retailer_scanned_at <= '2026-04-17 23:59:59'
GROUP BY si.status_retailer_id, sp.categories
),

```

```

NewCategoryActivations AS (
    SELECT
        c.retailer_id,
        c.category_id,
        c.current_total_boxes,
        mc.name as category_name,
        CASE WHEN c.current_total_boxes >= 5 THEN 1 ELSE 0 END as is_qualified
    FROM CurrentData c
    LEFT JOIN PrevData p

```

```

        ON c.retailer_id = p.retailer_id
        AND c.category_id = p.category_id
        JOIN master_categories mc ON mc.id = c.category_id
        WHERE (p.prev_total_boxes IS NULL OR p.prev_total_boxes < 5)
    )
    SELECT
        nca.retailer_id,
        md.shop_name,
        u.mobile_number,
        u.mobile_number as whatsapp_number,
        u.city,
        nca.category_id,
        nca.category_name,
        nca.current_total_boxes as boxes_scanned,
        nca.is_qualified
    FROM NewCategoryActivations nca
    JOIN users u ON u.id = nca.retailer_id
    LEFT JOIN mechanic_details md ON md.mechanic_id = u.id
    ORDER BY nca.retailer_id, nca.category_id;

```

#To Check Earning for Retailers

```

select sum(amount)
from wallet_transaction where reference_type IN ('cash_point','topup','referral_earning','incentive')
and user_id = 67417 and created_at BETWEEN '2025-03-01' and '2026-03-31';

```

#To Check cash redeemed for Retailers

```

select sum(amount)
from wallet_transaction where reference_type IN ('withdrawal','bonus_conversion')
and user_id = 67417 and created_at BETWEEN '2025-03-01' and '2026-03-31';

```

#To Check gift redeemed for Retailers

```

select sum(used_points)
from gift_orders
where status in ('Pending','Delivered')
and user_id = 67417;

```

#To Check TDS for Retailers

```
select * from withdrawal_request where user_id = 53756;
```

#To Check total_bonus for Retailers

```
SELECT SUM(points)
FROM bonus_transactions
WHERE type = 'credit' and reference_type is null
AND user_id = 67417
AND created_at BETWEEN '2025-03-01' AND '2026-03-31';
```

#To Check Total Balance for retailers

```
select sum(amount) from wallet_transaction where user_id = 53756 and created_at BETWEEN
'2025-06-01' AND '2026-03-31';
```

```
select sum(amount) from wallet_transaction where user_id = 53756;
```

```
SELECT SUM(amount)
FROM wallet_transaction
WHERE user_id = 53756
AND created_at >= '2025-06-01';
```

#To Check Bonus Balance for retailers

```
select sum(points) from bonus_transactions where user_id = 53756 and type = 'credit';
select sum(points) from bonus_transactions where user_id = 53756 and type = 'debit';
```

#Karnataka Toing Scheme 2026

```
SELECT
    si.distributer_id,
    si.id,
    u.`name`,
```

```
si.sku_code,  
si.lpn_number,  
si.uom,  
si.status_retailer_id,  
si.retailer_scanned_at,  
si.created_at,  
'Retailer' as role_name,  
sqpm.gride_type,  
sqpm.box_calculation_uom,    sp.product_name,  
COALESCE(wt.amount, 0) AS Earned_Units,  
sqpm.pcs_calculation_uom,  
csg.name
```

```
FROM sku_inventories si
```

```
JOIN users u
```

```
    ON u.id = si.status_retailer_id
```

```
JOIN sku_products sp
```

```
    ON sp.id = si.product_id
```

```
LEFT JOIN sku_qr_points_maps sqpm
```

```
    ON si.sku_code = sqpm.sku_code and sqpm.uom = si.uom
```

```
join category_sub_groups csg on
```

```
csg.id = sp.category_sub_group_id
```

```
JOIN scheme_wallet_transactions wt
```

```
    ON si.id = wt.reference_id and wt.reference_type = 'sku_scan'    and scheme_id = 3
```

```
WHERE u.user_role = 2;
```

#Distributor Daily Scanning Report

```
WITH user_info AS (
```

```
    SELECT u.id,
```

```
        COALESCE(m.shop_name, u.name) AS customer_name,
```

```
        u.pincodes, u.city,
```

```

        pam.city AS actual_city,
        s.sname AS state,
        z.name AS zone,
        u.user_role
FROM users u
LEFT JOIN mechanic_details m ON m.mechanic_id = u.id
LEFT JOIN state s ON s.id = u.state_id
LEFT JOIN zones z ON z.id = s.zone_id
LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode
WHERE u.user_role IN (2,5)
),

box_scans AS (
    SELECT reference_id AS skui_id, schemes.scheme_name,
           swt.amount AS earned_units, 'Scheme Scan' AS scan_type
    FROM scheme_wallet_transactions swt
    JOIN schemes ON schemes.id = swt.scheme_id
    WHERE swt.reference_type = 'sku_scan'
           AND swt.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'

    UNION ALL

    SELECT reference_id, NULL, gwt.amount, 'Gift Scan'
    FROM gift_scan_wallet_transactions gwt
    WHERE gwt.reference_type = 'sku_scan'
           AND gwt.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'

    UNION ALL

    SELECT reference_id, NULL, wst.amount, 'Winter Scan'
    FROM winter_scan_wallet_transactions wst
    WHERE wst.reference_type = 'sku_scan'
           AND wst.created_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59'
),

scan_data AS (
    SELECT skui.id,
           skui.lpn_number,
           COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
           sqpm.box_calculation_uom,
           sqpm.pcs_calculation_uom,
           DATE(skui.created_at) AS created_at,

```

```

skui.sku_code, skui.sku_description,
p.product_name, p.product_code,
p.season AS product_type,
mc.name AS category_name,
cg.name AS category_sub_group,
b.brand_name AS style,
d.name AS distributor,
d.sap_code,
dis_st.sname AS distributor_state,
DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
DATE(skui.retailer_scanned_at) AS retailer_scanned_on,

CASE
    WHEN WEEK(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at),0)
        - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at),
            INTERVAL
DAYOFMONTH(COALESCE(skui.retailer_scanned_at,skui.wholesaler_scanned_at))-1 DAY),0)+1 = 1 THEN
'Week 1'
    WHEN WEEK(...) = 2 THEN 'Week 2'
    WHEN WEEK(...) = 3 THEN 'Week 3'
    WHEN WEEK(...) = 4 THEN 'Week 4'
    ELSE 'Week 5'
END AS scan_week,

CASE
    WHEN skui.status_retailer_id IS NOT NULL AND skui.status_wholeseller_id IS NULL THEN
'Retailer'
    WHEN skui.status_wholeseller_id IS NOT NULL AND skui.status_retailer_id IS NULL THEN
'Wholesaler'
    ELSE 'Both'
END AS scan_by,

COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,
CASE WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer' ELSE 'Wholesaler' END AS
customer_type

FROM sku_inventories skui
JOIN sku_products p ON p.id = skui.product_id
LEFT JOIN master_categories mc ON mc.id = p.categories
LEFT JOIN category_sub_groups cg ON cg.id = p.category_sub_group_id
LEFT JOIN brands b ON b.id = p.brand
JOIN companies d ON d.id = skui.distributer_id

```

```

JOIN users du ON du.distributer_id = d.id AND du.user_role = 4
LEFT JOIN state dis_st ON dis_st.id = du.state_id
LEFT JOIN (
    SELECT sku_code, uom,
        MIN(box_calculation_uom) AS box_calculation_uom,
        MIN(pcs_calculation_uom) AS pcs_calculation_uom
    FROM sku_qr_points_maps
    GROUP BY sku_code, uom
) sqpm
ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')
AND sqpm.uom = skui.uom

WHERE (skui.retailer_scanned_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59')
    OR (skui.wholesaler_scanned_at BETWEEN '2025-02-28 00:00:00' AND '2025-08-31 23:59:59')
)

SELECT sd.*, ui.customer_name, ui.city, ui.pincode, ui.state, ui.zone, ui.actual_city,
    COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,
    bs.scheme_name,
    bs.earned_units,
    mp.points AS earned_point

FROM scan_data sd
LEFT JOIN user_info ui ON ui.id = sd.customer_id
LEFT JOIN box_scans bs ON bs.skui_id = sd.id
LEFT JOIN mypoints mp ON mp.sku_inventory_id = sd.id
    AND mp.users_id = sd.customer_id

WHERE sd.sap_code = 'D12IND2101';

```

```

WITH user_info AS (
    SELECT
        u.id,
        COALESCE(m.shop_name, u.name) AS customer_name,
        u.pincode,
        u.city,
        pam.city AS actual_city,

```

```

        s.sname AS state,
        z.name AS zone,
        u.user_role
FROM users u
LEFT JOIN mechanic_details m
    ON m.mechanic_id = u.id
LEFT JOIN state s
    ON s.id = u.state_id
LEFT JOIN zones z
    ON z.id = s.zone_id
LEFT JOIN pincode_address_maps pam
    ON pam.pincode = u.pincode
WHERE u.user_role IN (2,5)
),

```

```

box_scans AS (
    SELECT
        reference_id AS skui_id,
        schemes.scheme_name,
        swt.amount AS earned_units,
        'Scheme Scan' AS scan_type
    FROM scheme_wallet_transactions swt
    JOIN schemes
        ON schemes.id = swt.scheme_id
    WHERE swt.reference_type = 'sku_scan'
        AND swt.created_at >= '2025-02-01'
        AND swt.created_at < '2025-09-01'

```

UNION ALL

```

    SELECT
        reference_id,
        NULL,
        gwt.amount,
        'Gift Scan'
    FROM gift_scan_wallet_transactions gwt
    WHERE gwt.reference_type = 'sku_scan'
        AND gwt.created_at >= '2025-02-01'
        AND gwt.created_at < '2025-09-01'

```

UNION ALL

```

SELECT
    reference_id,
    NULL,
    wst.amount,
    'Winter Scan'
FROM winter_scan_wallet_transactions wst
WHERE wst.reference_type = 'sku_scan'
    AND wst.created_at >= '2025-02-01'
    AND wst.created_at < '2025-09-01'
),

scan_data AS (
    SELECT
        skui.id,
        skui.lpn_number,

        COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
        sqpm.box_calculation_uom,
        sqpm.pcs_calculation_uom,

        DATE(skui.created_at) AS created_at,

        skui.sku_code,
        skui.sku_description,

        p.product_name,
        p.product_code,
        p.season AS product_type,

        mc.name AS category_name,
        cg.name AS category_sub_group,
        b.brand_name AS style,

        d.name AS distributor,
        d.sap_code,
        dis_st.sname AS distributor_state,

        DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
        DATE(skui.retailer_scanned_at) AS retailer_scanned_on,

        --  Week Number (clean)
    (

```

```

WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
- WEEK(
    DATE_SUB(
        COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
        INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at)) -
1 DAY
    ), 0
    ) + 1
) AS week_number,

CASE
    WHEN skui.status_retailer_id IS NOT NULL
        AND skui.status_wholeseller_id IS NULL THEN 'Retailer'
    WHEN skui.status_wholeseller_id IS NOT NULL
        AND skui.status_retailer_id IS NULL THEN 'Wholesaler'
    ELSE 'Both'
END AS scan_by,

COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,

CASE
    WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer'
    ELSE 'Wholesaler'
END AS customer_type

FROM sku_inventories skui

JOIN sku_products p
    ON p.id = skui.product_id

LEFT JOIN master_categories mc
    ON mc.id = p.categories

LEFT JOIN category_sub_groups cg
    ON cg.id = p.category_sub_group_id

LEFT JOIN brands b
    ON b.id = p.brand

JOIN companies d
    ON d.id = skui.distributer_id

```

```

-- ☒ fixed join (no data loss)
LEFT JOIN users du
    ON du.distributer_id = d.id
    AND du.user_role = 4

LEFT JOIN state dis_st
    ON dis_st.id = du.state_id

LEFT JOIN (
    SELECT
        sku_code,
        uom,
        MIN(box_calculation_uom) AS box_calculation_uom,
        MIN(pcs_calculation_uom) AS pcs_calculation_uom
    FROM sku_qr_points_maps
    GROUP BY sku_code, uom
) sqpm
    ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')
    AND sqpm.uom = skui.uom

-- ☒ correct date filter
WHERE
    COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at) >= '2025-02-01'
    AND COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at) < '2025-09-01'
)

SELECT
    sd.*,

-- ☒ final week label
CASE
    WHEN sd.week_number = 1 THEN 'Week 1'
    WHEN sd.week_number = 2 THEN 'Week 2'
    WHEN sd.week_number = 3 THEN 'Week 3'
    WHEN sd.week_number = 4 THEN 'Week 4'
    ELSE 'Week 5'
END AS scan_week,

ui.customer_name,
ui.city,
ui.pincode,
ui.state,

```

```

    ui.zone,
    ui.actual_city,

    COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,
    bs.scheme_name,
    bs.earned_units,

    mp.points AS earned_point

FROM scan_data sd

LEFT JOIN user_info ui
    ON ui.id = sd.customer_id

LEFT JOIN box_scans bs
    ON bs.skui_id = sd.id

LEFT JOIN mypoints mp
    ON mp.sku_inventory_id = sd.id
    AND mp.users_id = sd.customer_id

WHERE sd.sap_code = 'D12IND2101';

```

– Karnataka Toing Scheme

```

SELECT
    si.distributer_id,
    si.id,
    u.`name`,
    si.sku_code,
    si.lpn_number,
    si.uom,
    si.status_retailer_id,
    si.retailer_scanned_at,
    si.created_at,
    'Retailer' as role_name,
    sqpm.gride_type,
    sqpm.box_calculation_uom, sp.product_name,
    COALESCE(wt.amount, 0) AS Earned_Units,

```

```

    sqpm.pcs_calculation_uom,
    csg.name

FROM sku_inventories si

JOIN users u
    ON u.id = si.status_retailer_id
JOIN sku_products sp
    ON sp.id = si.product_id
LEFT JOIN sku_qr_points_maps sqpm
    ON si.sku_code = sqpm.sku_code and sqpm.uom = si.uom
join category_sub_groups csg on
    csg.id = sp.category_sub_group_id

JOIN scheme_wallet_transactions wt
    ON si.id = wt.reference_id and wt.reference_type = 'sku_scan' and scheme_id = 3
WHERE u.user_role = 2;

```

#Growth Scheme Report for bonus points

```

SELECT
    wt.user_id,
    SUM(wt.amount) AS Earning_points,

    u.name,
    u.mobile_number,

    s.sname AS state,
    md.shop_name

FROM wallet_transaction wt

JOIN users u
    ON u.id = wt.user_id

```

```
JOIN state s
  ON u.state_id = s.id

JOIN mechanic_details md
  ON md.mechanic_id = u.id

WHERE wt.created_at >= '2026-02-01'
  AND wt.created_at < '2026-04-01'
  AND wt.reference_type = 'incentive'
```

```
GROUP BY
  wt.user_id,
  u.name,
  u.mobile_number,
  s.sname,
  md.shop_name;
```

#Growth Scheme feb to mar all required columns

```
SELECT
  u.id,
  u.state_id,
  u.city,
  u.user_role,
  u.pincode,
  m.monthly_average,

  COALESCE(us.Unit_Scanned, 0) AS Unit_Scanned,
  COALESCE(mp.Points, 0) AS Points,
  COALESCE(bp.Bonus_Points, 0) AS Bonus_Points,
  COALESCE(gp.Growth_Points, 0) AS Growth_Points,

  (
    COALESCE(bp.Bonus_Points, 0) +
    COALESCE(gp.Growth_Points, 0)
  ) AS Total_Points_Earned
```

```
FROM users u
```

```

JOIN monthly_averages m
    ON u.id = m.user_id

-- Unit Scanned
LEFT JOIN (
    SELECT
        si.status_retailer_id ,
        SUM(sq.box_calculation_uom) AS Unit_Scanned
    FROM sku_inventories si
    JOIN sku_qr_points_maps sq
    ON si.uom = sq.uom
    AND si.sku_code = sq.sku_code
    GROUP BY si.status_retailer_id
) us
    ON u.id = us.status_retailer_id

-- Total Points
LEFT JOIN (
    SELECT
        users_id,
        SUM(points) AS Points
    FROM mypoints
    WHERE created_at >= '2026-03-01'
    AND created_at < '2026-04-01'
    GROUP BY users_id
) mp
    ON u.id = mp.users_id

-- Bonus Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Bonus_Points
    FROM wallet_transaction
    WHERE reference_type = 'topup'
    AND reference_id REGEXP '^(13|19|20|21|22)'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) bp
    ON u.id = bp.user_id

```

```
-- Growth Points
LEFT JOIN (
    SELECT
        user_id,
        SUM(amount) AS Growth_Points
    FROM wallet_transaction
    WHERE reference_type = 'incentive'
    AND created_at >= '2026-04-01'
    AND created_at < '2026-05-01'
    GROUP BY user_id
) gp
    ON u.id = gp.user_id

WHERE u.user_role = 2;
```

#Toing_Scheme_Report

#Retailer_Month_wise_Ipn_number and Box_count

```
SELECT
    u.id,
    c.sap_code,
    md.shop_name,
    u.primary_distributor_id,
    u.pan_card,
    u.adhaar,
    u.mobile_number,
    u.email,
    s.sname AS State_name,
    u.district,
    u.city,
    u.pincode,

    /* Apr 2025 */
    COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-04' THEN si.lpn_number
    END) AS Apr_2025_lpn_count,
    SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-04'
        THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
    END)
        ELSE 0 END) AS Apr_2025_uom_count,

    /* May 2025 */
    COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-05' THEN si.lpn_number
    END) AS May_2025_lpn_count,
    SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-05'
        THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
    END)
        ELSE 0 END) AS May_2025_uom_count,
```

/* Jun 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-06' THEN si.lpn_number
END) AS Jun_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-06'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Jun_2025_uom_count,
```

/* Jul 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-07' THEN si.lpn_number
END) AS Jul_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-07'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Jul_2025_uom_count,
```

/* Aug 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-08' THEN si.lpn_number
END) AS Aug_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-08'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Aug_2025_uom_count,
```

/* Sep 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-09' THEN si.lpn_number
END) AS Sep_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-09'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Sep_2025_uom_count,
```

/* Oct 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-10' THEN si.lpn_number
END) AS Oct_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-10'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Oct_2025_uom_count,
```

/* Nov 2025 */

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-11' THEN si.lpn_number
END) AS Nov_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-11'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Nov_2025_uom_count,
```

```
/* Dec 2025 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-12' THEN si.lpn_number
END) AS Dec_2025_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2025-12'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Dec_2025_uom_count,
```

```
/* Jan 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-01' THEN si.lpn_number
END) AS Jan_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-01'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Jan_2026_uom_count,
```

```
/* Feb 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-02' THEN si.lpn_number
END) AS Feb_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-02'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Feb_2026_uom_count,
```

```
/* Mar 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-03' THEN si.lpn_number
END) AS Mar_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-03'
THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
ELSE 0 END) AS Mar_2026_uom_count,
```

```
/* Apr 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-04' THEN si.lpn_number
END) AS Apr_2026_lpn_count,
```

```
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-04'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Apr_2026_uom_count,
```

```
/* May 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-05' THEN si.lpn_number
END) AS May_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-05'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS May_2026_uom_count,
```

```
/* Jun 2026 */
```

```
COUNT(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-06' THEN si.lpn_number
END) AS Jun_2026_lpn_count,
SUM(CASE WHEN DATE_FORMAT(si.retailer_scanned_at,'%Y-%m')='2026-06'
      THEN COALESCE(NULLIF(sqpm.box_calculation_uom,0),CASE WHEN si.uom='B5' THEN 0.5 ELSE 1
END)
      ELSE 0 END) AS Jun_2026_uom_count,
```

```
MAX(si.retailer_scanned_at) AS last_scan_date
```

```
FROM users u
```

```
LEFT JOIN companies c
  ON c.id = u.primary_distributor_id
```

```
LEFT JOIN sku_inventories si
  ON si.status_retailer_id = u.id
```

```
LEFT JOIN (
  SELECT
    sku_code,
    uom,
    MIN(box_calculation_uom) AS box_calculation_uom
  FROM sku_qr_points_maps
  GROUP BY sku_code, uom
) sqpm
  ON sqpm.sku_code = REPLACE(si.sku_code, ' ', '')
 AND sqpm.uom = si.uom
```

```
JOIN mechanic_details md
  ON md.mechanic_id = u.id
```

```
LEFT JOIN state s
  ON s.id = u.state_id
```

```
WHERE u.user_role = 2
```

```
GROUP BY
  u.id,
  c.sap_code,
  md.shop_name,
  u.primary_distributor_id,
  u.pan_card,
  u.adhaar,
  u.mobile_number,
  u.email,
  s.sname,
  u.district,
  u.city,
  u.pincode;
```

#LPN Dump Data

```
SELECT
  lpn_number,
  sku_code,
  sku_description,
  carton_no,
  uom,
  invoice_number,
  invoice_date,
  warehouse,
```

sources,
retailer_scanned_at,
wholesaler_scanned_at,
sap_code,

CASE
 WHEN is_callback = 0 THEN 'Distributor Generator'
 WHEN is_callback = 1 THEN 'API Scan'
 WHEN is_callback = 2 THEN 'CSV Uploaded'
 ELSE 'Unknown'
END AS lpn_source,

CASE
 WHEN status_retailer_id IS NOT NULL THEN 'YES'
 ELSE 'NO'
END AS retailer_scan_status,

CASE
 WHEN wholesaler_scanned_at IS NOT NULL THEN 'YES'
 ELSE 'NO'
END AS wholesaler_scan_status,

CASE
 WHEN is_active = 0 THEN 'YES'
 ELSE 'NO'
END AS distributor_deactivated_status

FROM sku_inventories
join companies on
companies.id = sku_inventories.distributer_id;

#Retailer,Wholesaller,Distributor may month earning

```
SELECT
    u.id,
    r.name AS user_type,
    u.name as User_name,
    u.mobile_number,
    md.shop_name,

    SUM(
        CASE
            WHEN wt.created_at BETWEEN '2026-05-01' AND '2026-05-31'
            AND wt.reference_type IN ('withdrawal','bonus_conversion')
            THEN wt.amount
            ELSE 0
        END
    ) AS Redemption,

    SUM(
        CASE
            WHEN wt.created_at BETWEEN '2026-05-01' AND '2026-05-31'
            AND wt.reference_type IN ('cash_point','topup','referral_earning','incentive')
            THEN wt.amount
            ELSE 0
        END
    ) AS Total_Earning,

    SUM(
        CASE
            WHEN wt.created_at < '2026-05-01'
            THEN wt.amount
            ELSE 0
        END
    ) AS Open_Balance,
    sum(wt.amount) as Cash_Balance
```

```
FROM users u
JOIN wallet_transaction wt
      ON wt.user_id = u.id
JOIN role r
      ON r.id = u.user_role

join mechanic_details md on
md.mechanic_id = u.id

WHERE u.user_role IN (2,5,4)

GROUP BY
      wt.user_id,
      r.name,
      u.name,
      u.mobile_number,
      md.shop_name;
```

#Distributor Unscan monthly Core Balance report

```
SELECT
      u.distributer_id,
      dqwt.created_at,
      si.lpn_number,
      si.sku_description,
      si.sku_code,
      sp.product_code,
```

```
sp.size,  
si.uom,  
sq.box_calculation_uom,  
sq.gride_type,  
c.sap_code,
```

```
SUM(  
CASE  
    WHEN dqwt.reference_type IN ('sku_scan')  
        AND sp.season = 'core'  
    THEN dqwt.amount  
    ELSE 0  
END  
) AS core_Earning,
```

```
SUM(dqwt.amount) AS Balance,
```

```
SUM(  
CASE  
    WHEN dqwt.reference_type IN ('topup')  
    THEN dqwt.amount  
    ELSE 0  
END  
) AS Topup_Earning,
```

```
SUM(  
CASE  
    WHEN dqwt.reference_type IN ('credit_note') -- abs  
    THEN dqwt.amount  
    ELSE 0  
END  
) AS credit_note
```

```
FROM deactivated_qr_wallet_transactions dqwt
```

```
JOIN users u  
    ON u.id = dqwt.user_id
```

```
JOIN sku_inventories si  
    ON si.id = dqwt.reference_id
```

```
JOIN sku_products sp  
    ON sp.id = si.product_id
```

```
join sku_qr_points_maps sq on  
sq.sku_code = si.sku_code
```

```
join companies c on  
c.id = si.distributer_id
```

```
WHERE dqwt.created_at >= '2026-04-01' and dqwt.created_at < '2026-06-13'  
AND u.user_role = 4 and u.distributer_id = 6147 and dqwt.season = 'core'  
GROUP BY u.distributer_id,  
        dqwt.created_at,  
        si.lpn_number,  
        si.sku_description,  
        si.sku_code,  
        sp.product_code,  
        sp.size,  
        si.uom,  
        sq.box_calculation_uom,  
        sq.gride_type,  
        c.sap_code;
```

GIFT Order Redemption

```
SELECT  
    og.order_id,  
    og.user_id AS customer_id,  
    og.shop_name,  
    u.mobile_number,  
    og.status,  
    og.used_points,  
    og.created_at,  
    og.quantity,  
    gc.`name` as Gift_Name,  
    og.order_date,
```

```

s.sname as State_Name,
gc.gift_value,

SUM(
    CASE
        WHEN wt.created_at <= '2026-06-28' THEN wt.amount
        ELSE 0
    END
) AS `Wallet Balance Before Redemption`,

SUM(
    CASE
        WHEN bt.created_at <= '2026-06-28' THEN bt.points
        ELSE 0
    END
) AS `Bonus Points Before Redemption`

FROM gift_orders og
JOIN users u
    ON u.id = og.user_id
JOIN wallet_transaction wt
    ON wt.user_id = og.user_id
JOIN bonus_transactions bt
    ON bt.user_id = og.user_id
join gift_cards gc on
gc.id = og.gift_card_id
join state s on
s.id = u.state_id
WHERE og.created_at BETWEEN '2026-06-29' AND '2026-07-02'
GROUP BY
    og.order_id,
    og.user_id,
    og.shop_name,
    u.mobile_number,
    og.status,
    og.used_points,
    og.created_at,
    og.quantity,
    gc.`name`,
    og.order_date,
    s.sname,
    gc.gift_value;

```

#Distributor wise Retailer Data

```
WITH user_info AS (  
  SELECT u.id,  
         COALESCE(m.shop_name, u.name) AS customer_name,  
         u.pincode, u.city,  
         pam.city AS actual_city,  
         s.sname AS state,  
         z.name AS zone,  
         u.user_role  
  FROM users u  
  LEFT JOIN mechanic_details m ON m.mechanic_id = u.id  
  LEFT JOIN state s ON s.id = u.state_id  
  LEFT JOIN zones z ON z.id = s.zone_id  
  LEFT JOIN pincode_address_maps pam ON pam.pincode = u.pincode  
  WHERE u.user_role IN (2,5)  
)  
  
box_scans AS (  
  SELECT reference_id AS skui_id, schemes.scheme_name,  
         swt.amount AS earned_units, 'Scheme Scan' AS scan_type  
  FROM scheme_wallet_transactions swt  
  JOIN schemes ON schemes.id = swt.scheme_id  
  WHERE swt.reference_type = 'sku_scan'  
         AND swt.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'  
  UNION ALL  
  SELECT reference_id, NULL, gwt.amount, 'Gift Scan'  
  FROM gift_scan_wallet_transactions gwt  
  WHERE gwt.reference_type = 'sku_scan'  
         AND gwt.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'  
  UNION ALL  
  SELECT reference_id, NULL, wst.amount, 'Winter Scan'
```

```

FROM winter_scan_wallet_transactions wst
WHERE wst.reference_type = 'sku_scan'
      AND wst.created_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59'
),
scan_data AS (
  SELECT skui.id,
         skui.lpn_number,
         COALESCE(skui.uom, sqpm.uom) AS resolved_uom,
         sqpm.box_calculation_uom,
         sqpm.pcs_calculation_uom,
         DATE(skui.created_at) AS created_at,
         skui.sku_code, skui.sku_description,
         p.product_name, p.product_code,
         p.season AS product_type,
         mc.name AS category_name,
         cg.name AS category_sub_group,
         b.brand_name AS style,
         d.name AS distributor,
         d.sap_code,
         dis_st.sname AS distributor_state,
         DATE(skui.wholesaler_scanned_at) AS wholesaler_scanned_on,
         DATE(skui.retailer_scanned_at) AS retailer_scanned_on,
         CASE
           WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                - 1 DAY), 0) + 1 = 1 THEN 'Week 1'
           WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                - 1 DAY), 0) + 1 = 2 THEN 'Week 2'
           WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                - 1 DAY), 0) + 1 = 3 THEN 'Week 3'
           WHEN WEEK(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at), 0)
                - WEEK(DATE_SUB(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at),
                                INTERVAL DAYOFMONTH(COALESCE(skui.retailer_scanned_at, skui.wholesaler_scanned_at))
                                - 1 DAY), 0) + 1 = 4 THEN 'Week 4'
           ELSE 'Week 5'
         END AS scan_week,
         CASE

```

```

        WHEN skui.status_retailer_id IS NOT NULL AND skui.status_wholeseller_id IS NULL THEN
'Retailer'
        WHEN skui.status_wholeseller_id IS NOT NULL AND skui.status_retailer_id IS NULL THEN
'Wholesaler'
        ELSE 'Both'
    END AS scan_by,
    COALESCE(skui.status_retailer_id, skui.status_wholeseller_id) AS customer_id,
    CASE WHEN skui.status_retailer_id IS NOT NULL THEN 'Retailer' ELSE 'Wholesaler' END AS
customer_type
FROM sku_inventories skui
JOIN sku_products p ON p.id = skui.product_id
LEFT JOIN master_categories mc ON mc.id = p.categories
LEFT JOIN category_sub_groups cg ON cg.id = p.category_sub_group_id
LEFT JOIN brands b ON b.id = p.brand
JOIN companies d ON d.id = skui.distributer_id
JOIN users du ON du.distributer_id = d.id AND du.user_role = 4
LEFT JOIN state dis_st ON dis_st.id = du.state_id
LEFT JOIN (
    SELECT sku_code, uom,
        MIN(box_calculation_uom) AS box_calculation_uom,
        MIN(pcs_calculation_uom) AS pcs_calculation_uom
    FROM sku_qr_points_maps
    GROUP BY sku_code, uom
) sqpm
ON sqpm.sku_code = REPLACE(skui.sku_code, ' ', '')
AND sqpm.uom = skui.uom
WHERE (skui.retailer_scanned_at BETWEEN '2025-12-01 00:00:00' AND '2026-01-31 23:59:59')
)
SELECT sd.*, ui.customer_name, ui.city, ui.pincode, ui.state, ui.zone, ui.actual_city,
    COALESCE(bs.scan_type, 'Cash Point Scan') AS scan_type,
    bs.scheme_name,
    bs.earned_units,
    mp.points AS earned_point
FROM scan_data sd
LEFT JOIN user_info ui ON ui.id = sd.customer_id
LEFT JOIN box_scans bs ON bs.skui_id = sd.id
LEFT JOIN mypoints mp ON mp.sku_inventory_id = sd.id
    AND mp.users_id = sd.customer_id
WHERE sd.sap_code = 'D12RTM0901';

```

New Categories Specific SE WISE Report

```
SELECT
  sk.status_retailer_id AS retailer_id,
  c.id AS distributor_id,
  m.name AS category,
  count(sk.lpn_number) as scan_count,
  SUM(
    CASE
      WHEN sqpm.box_calculation_uom IS NOT NULL
        AND sqpm.box_calculation_uom > 0
      THEN sqpm.box_calculation_uom
      WHEN sk.uom = 'B5' THEN 0.5
      ELSE 1
    END
  ) AS current_category_box_count,
  sqpm.uom,
```

```

    u.name,
    u.mobile_number,
    md.shop_name
FROM sku_inventories sk
JOIN users u
    ON u.id = sk.status_retailer_id
JOIN sku_products sp
    ON sp.id = sk.product_id
JOIN master_categories m
    ON m.id = sp.categories
JOIN executive_distributor_mappings edm
    ON edm.distributor_id = u.primary_distributor_id
JOIN sku_qr_points_maps sqpm
    ON sqpm.sku_code = sk.sku_code
    AND sqpm.uom = sk.uom
JOIN mechanic_details md
    ON md.mechanic_id = u.id
join companies c on
c.id = u.primary_distributor_id
WHERE u.user_role = 2
    AND edm.executive_id = 53478
    AND sk.retailer_scanned_at >= '2026-04-01'
    AND sk.retailer_scanned_at < '2026-05-01'
    AND NOT EXISTS (
        SELECT 1
        FROM sku_inventories sk2
        JOIN sku_products sp2
            ON sp2.id = sk2.product_id
        WHERE sk2.status_retailer_id = sk.status_retailer_id
            AND sp2.categories = sp.categories
            AND sk2.retailer_scanned_at < '2026-03-01'
    )
GROUP BY
    sk.status_retailer_id,
    sk.distributer_id,
    m.name,
    sk.uom,
    u.name,
    u.mobile_number,
    md.shop_name
ORDER BY
    sk.status_retailer_id,

```

m.name;